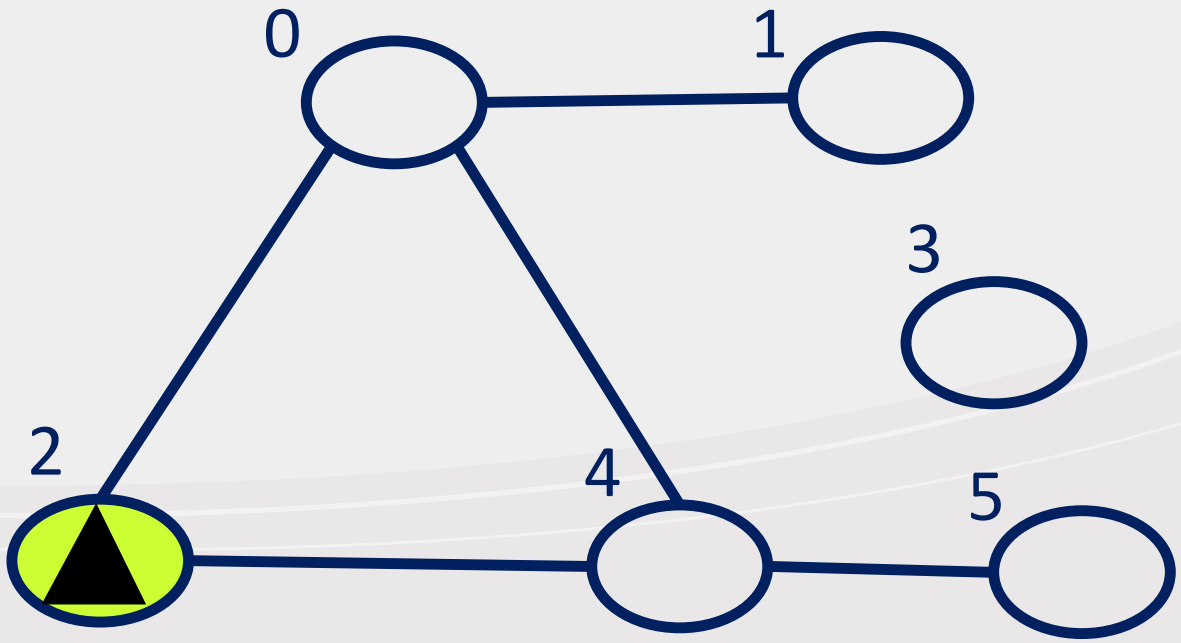
Graph Traversals and their usage

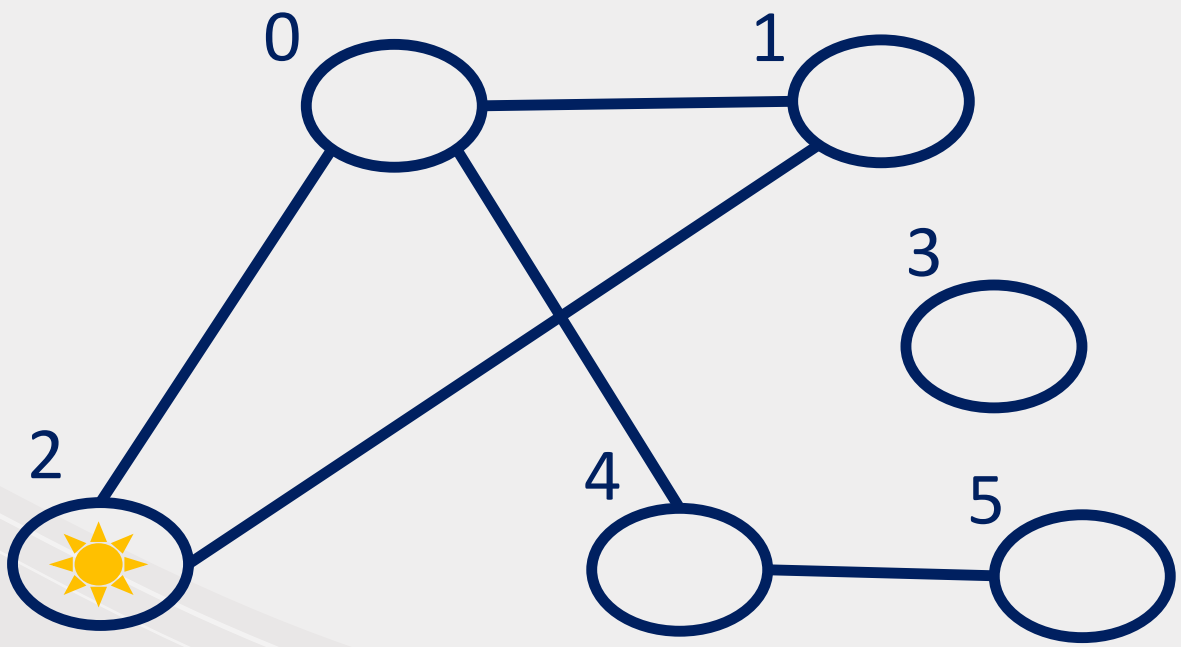
Breadth first search (BFS)

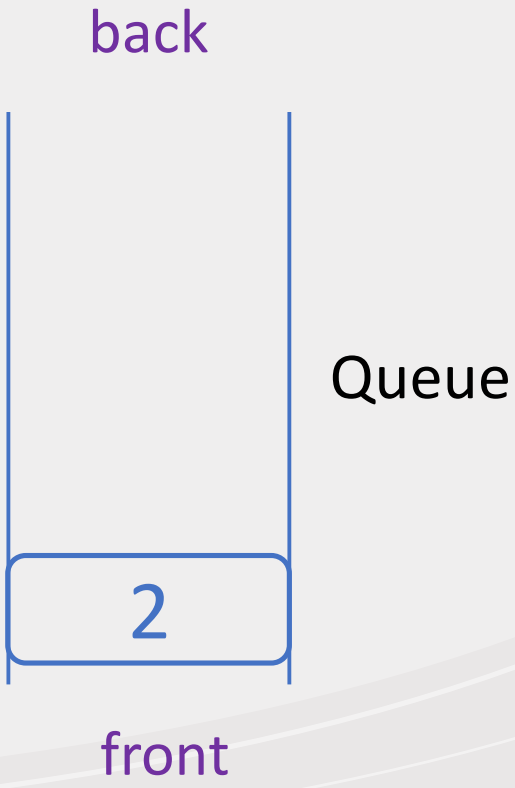
- Explores all nodes at the present depth / distance prior to moving on to the nodes at the next depth / distance level
- A main procedure for many graph algorithms

- Traversal: 2

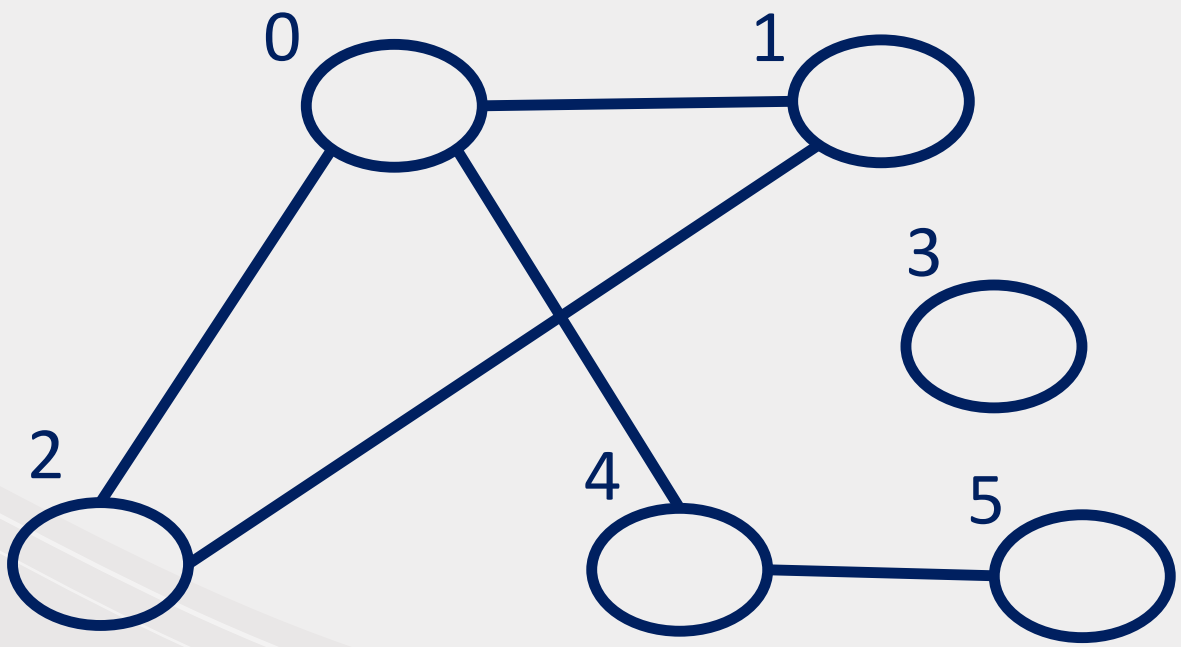


Breadth first search





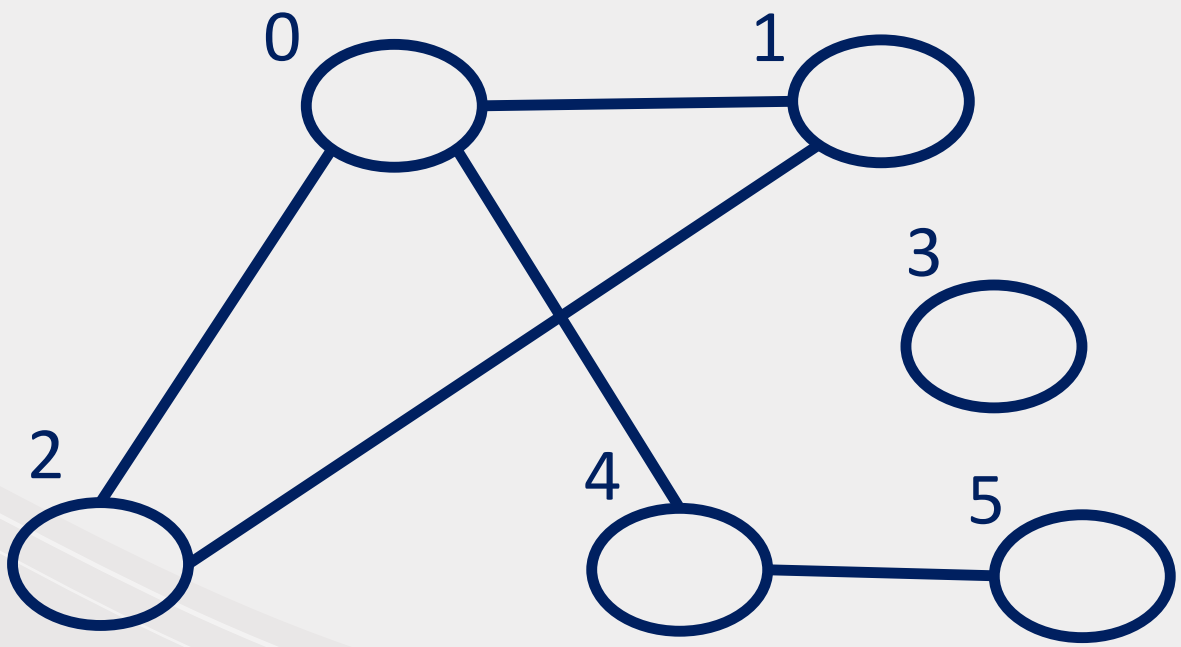
Breadth first search



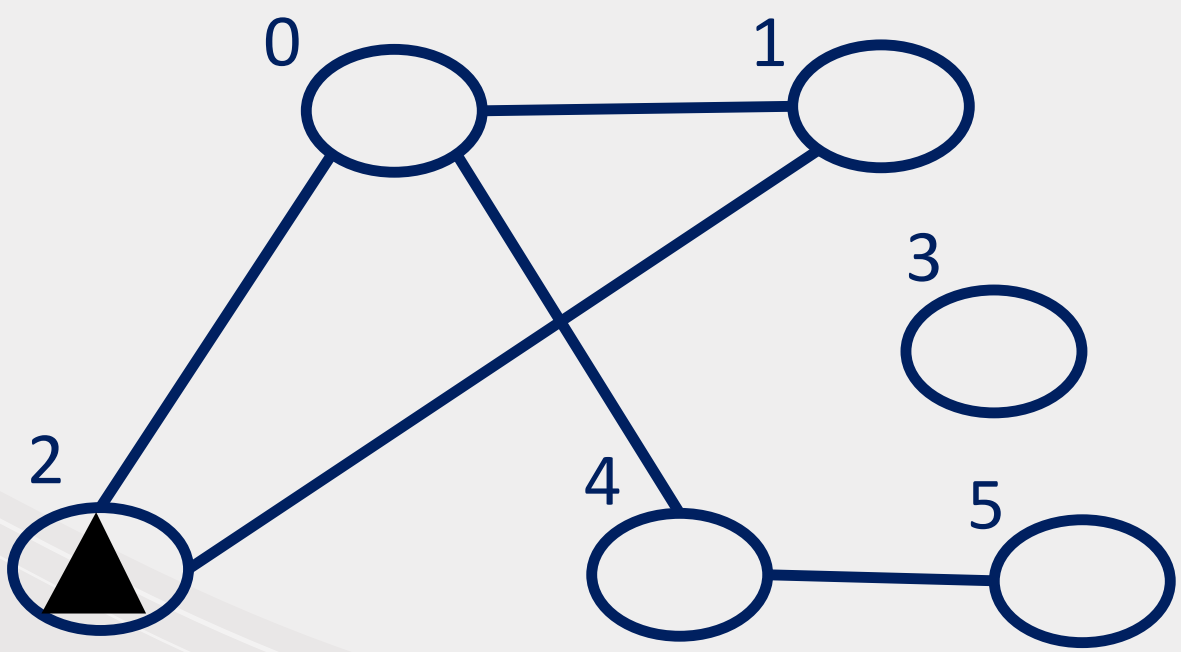
Is the queue empty?

Breadth first search

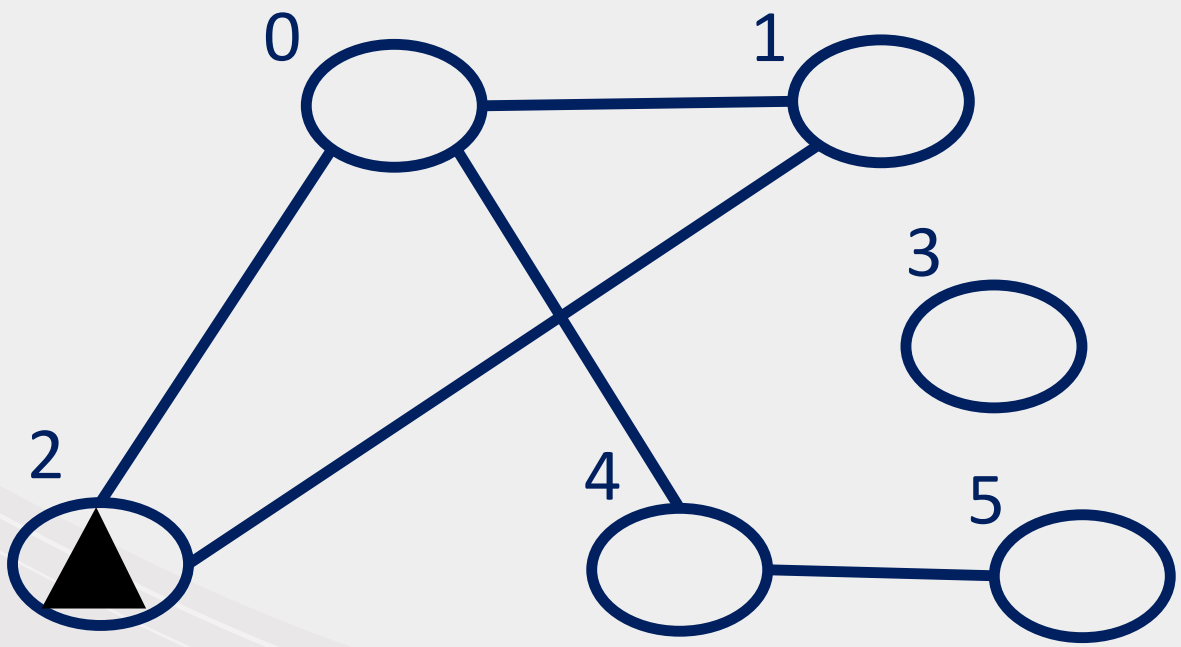
No! => Dequeue the node and make it current



Breadth first search

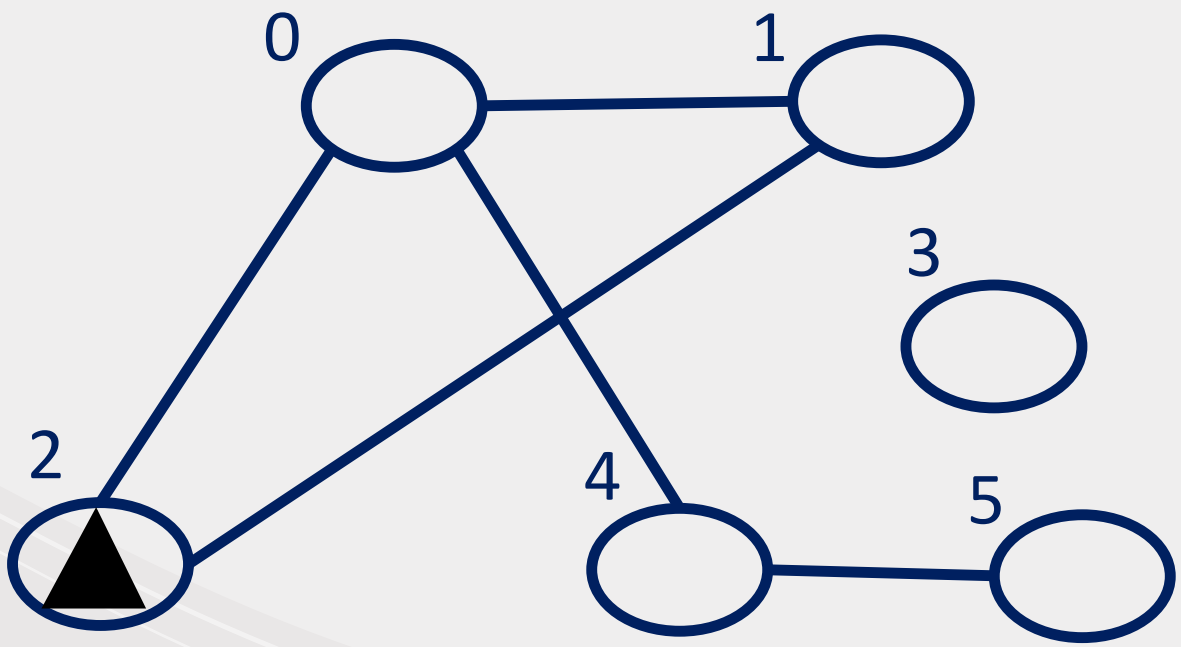


Breadth first search



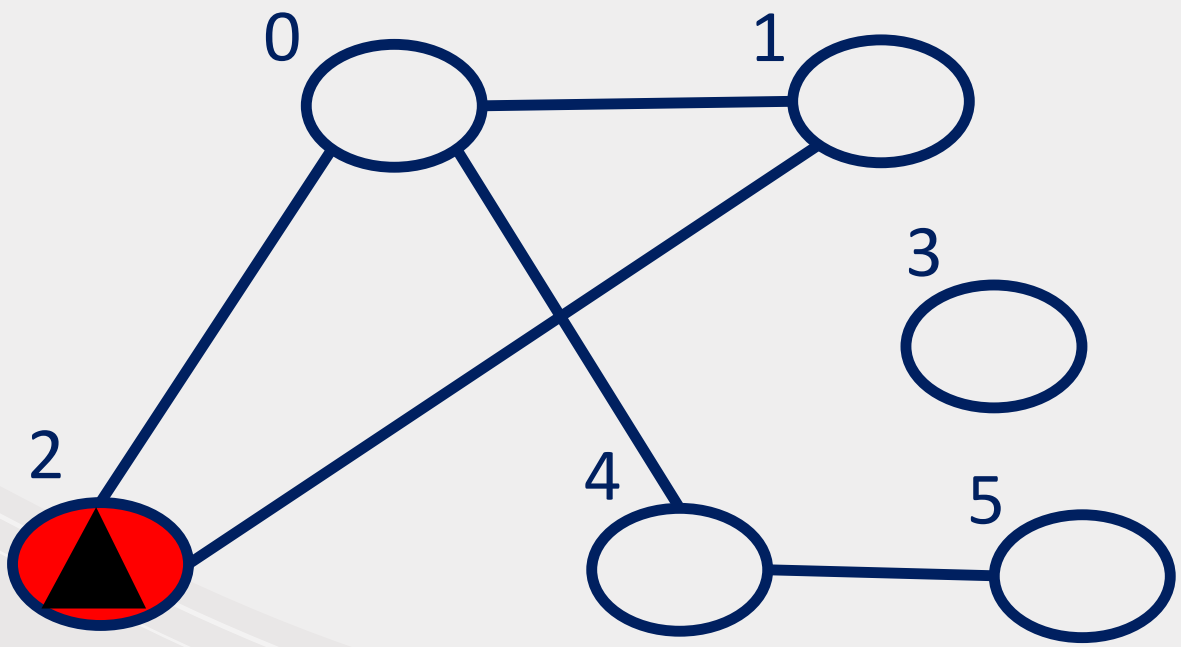
Is the node visited?

Breadth first search



No! => Make it visited and add to the traversal

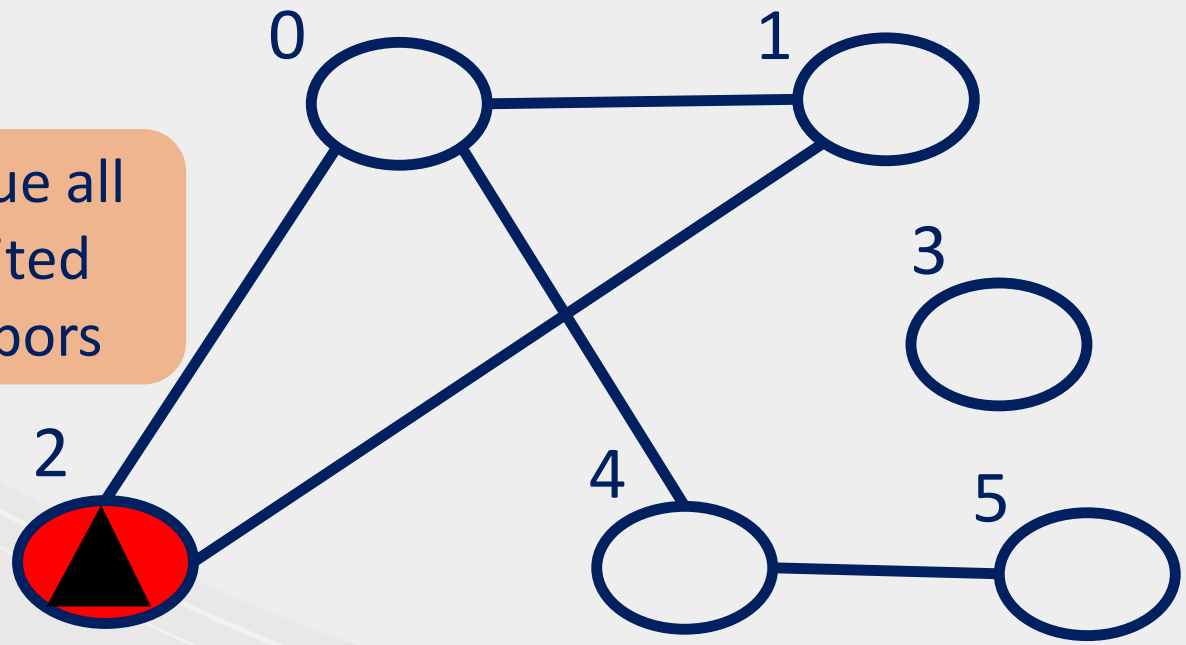
Breadth first search



Traversal: 2

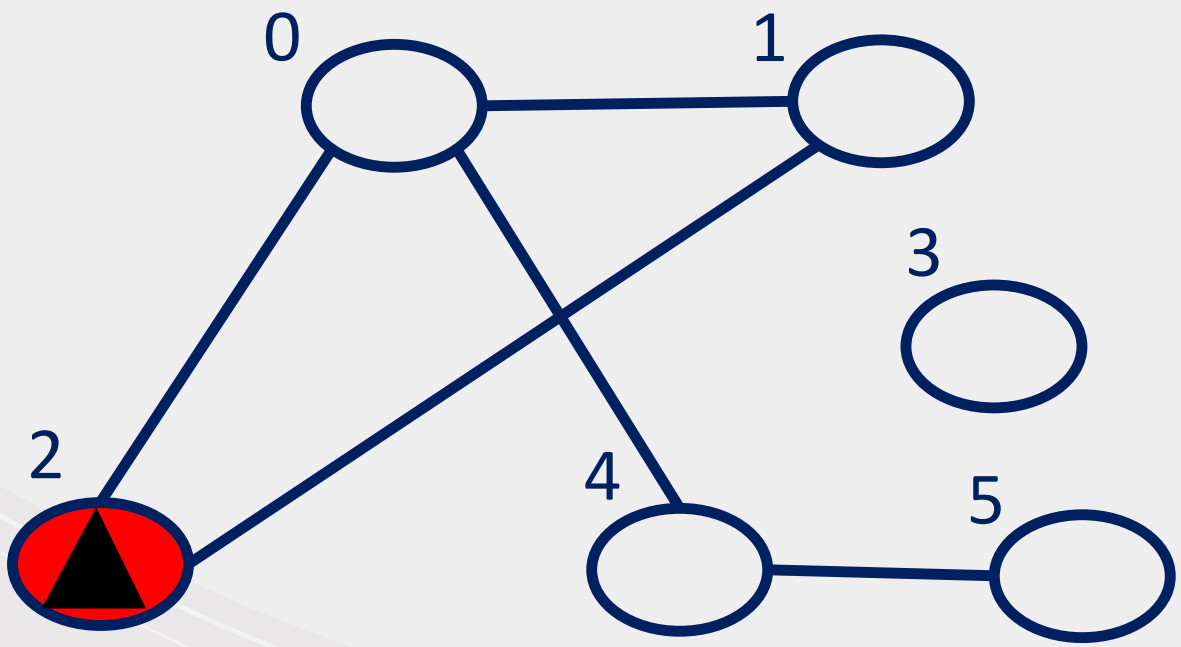
Breadth first search

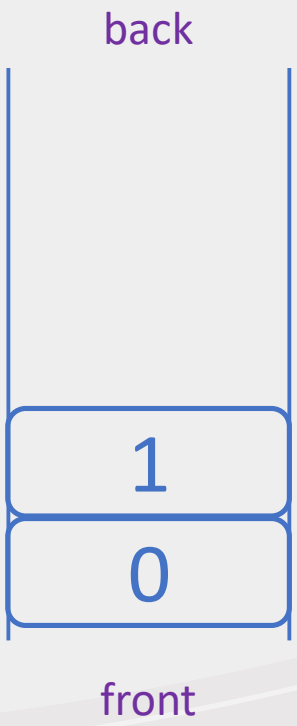
Enqueue all unvisited neighbors



Traversal: 2

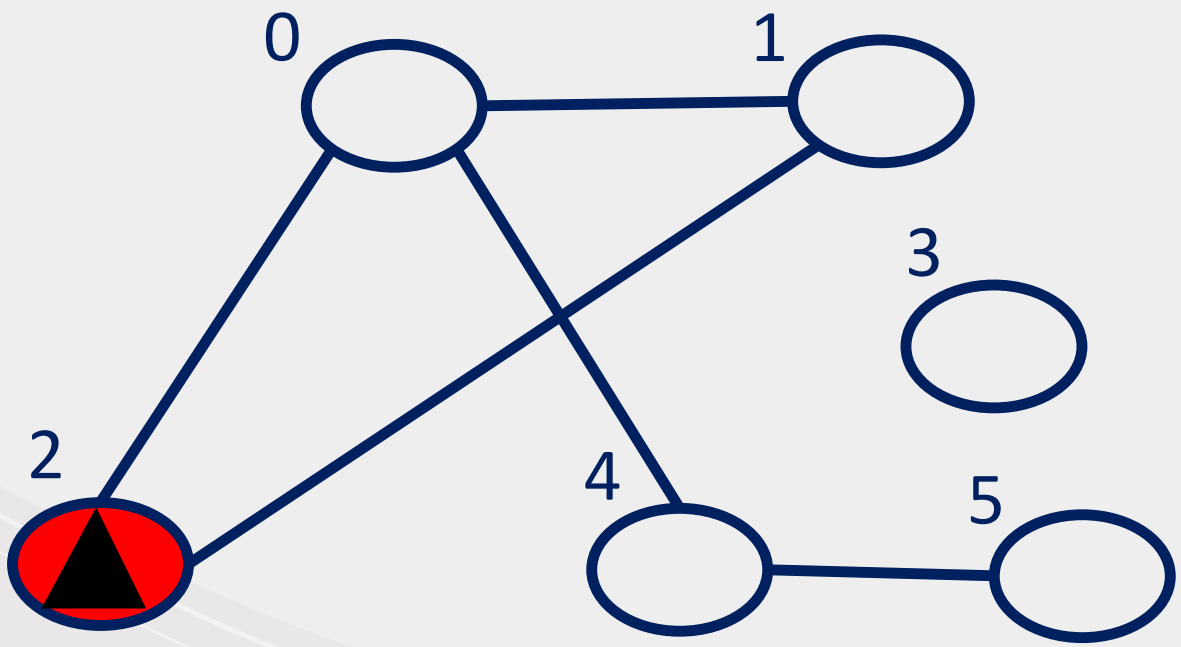
Breadth first search



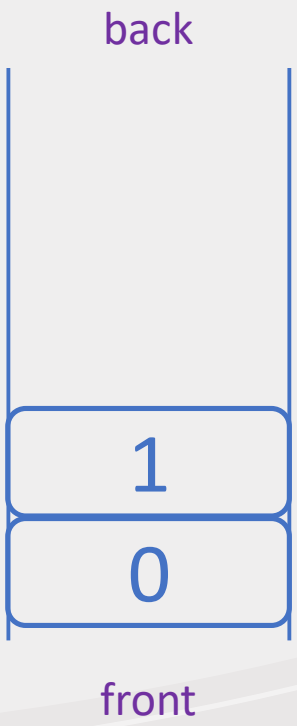


Traversal: 2

Breadth first search



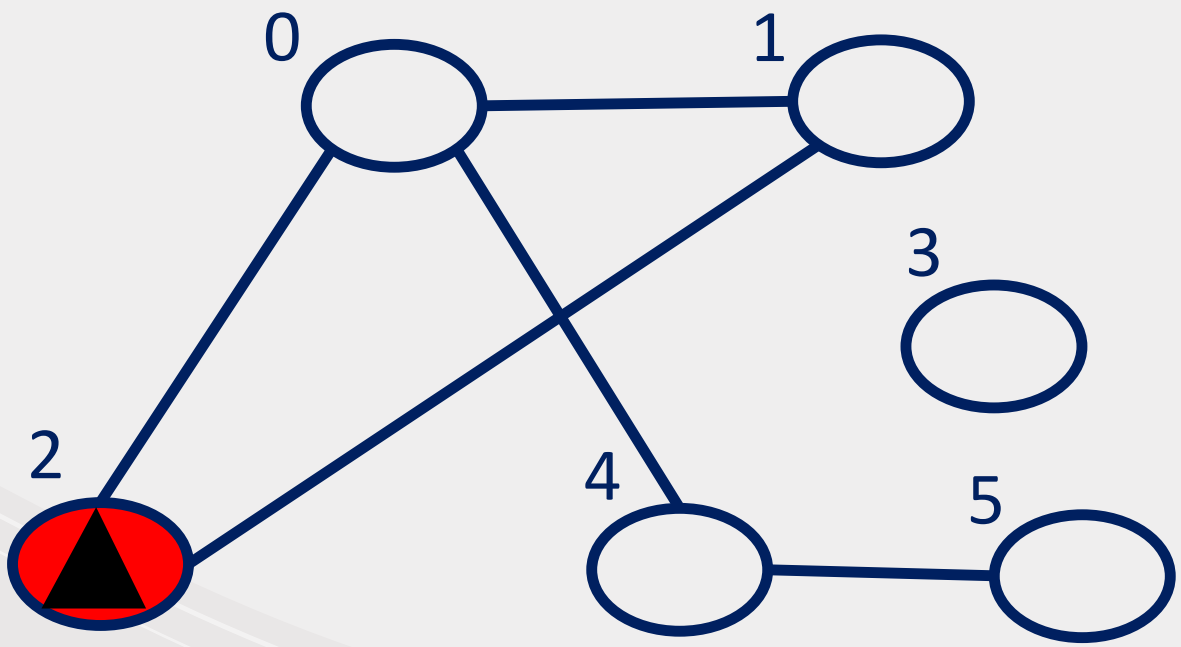
Is the queue empty?

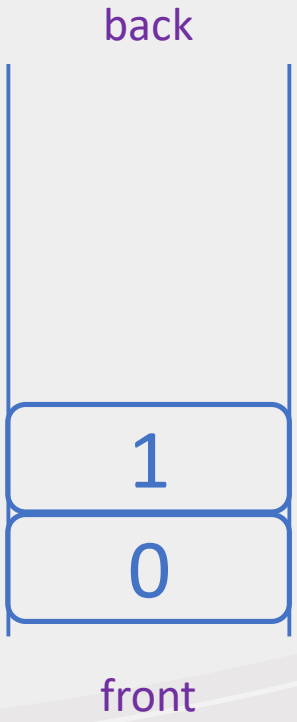


Traversal: 2

Breadth first search

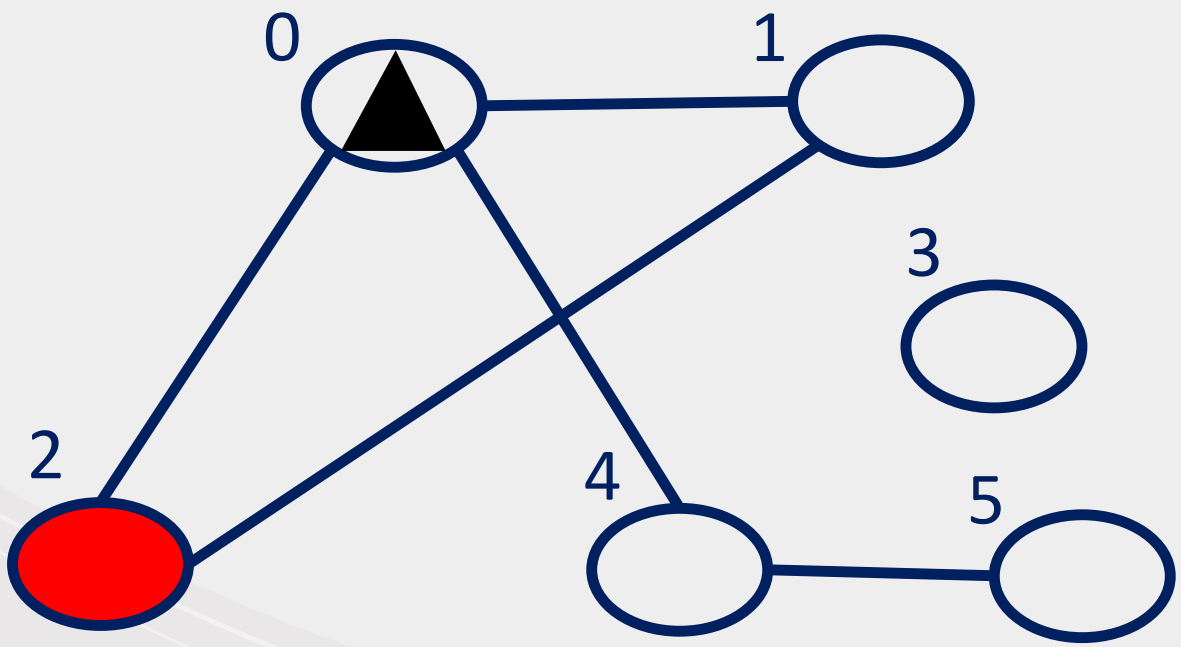
No! => Dequeue the node and make it current





Traversal: 2

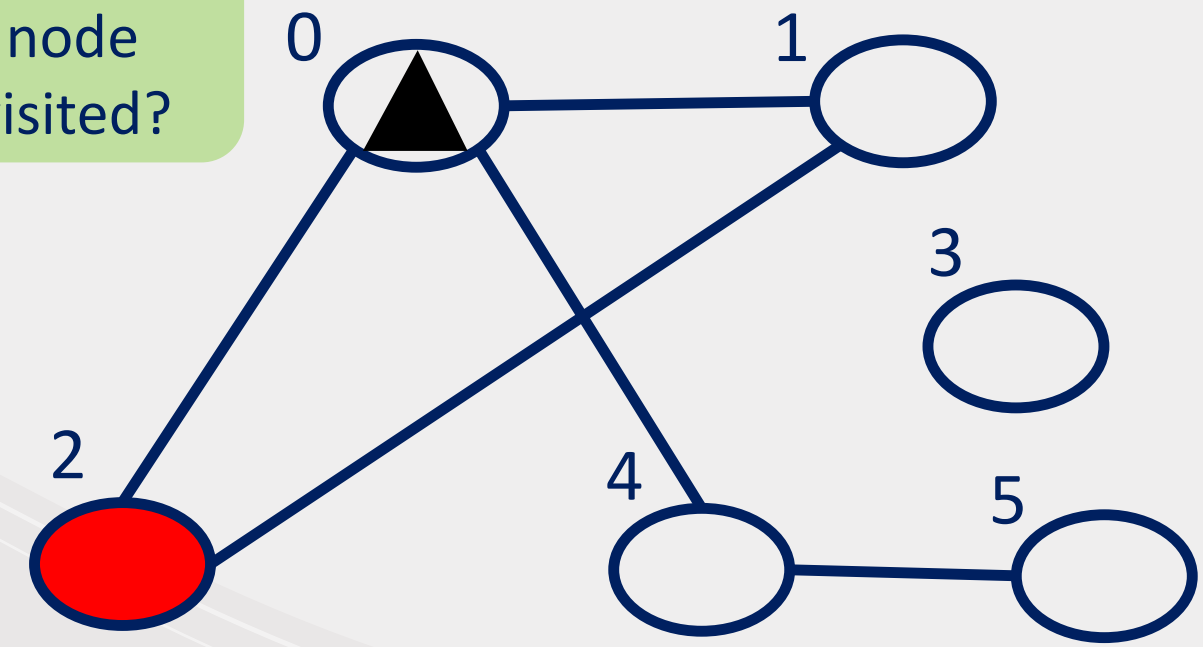
Breadth first search



Traversal: 2

Breadth first search

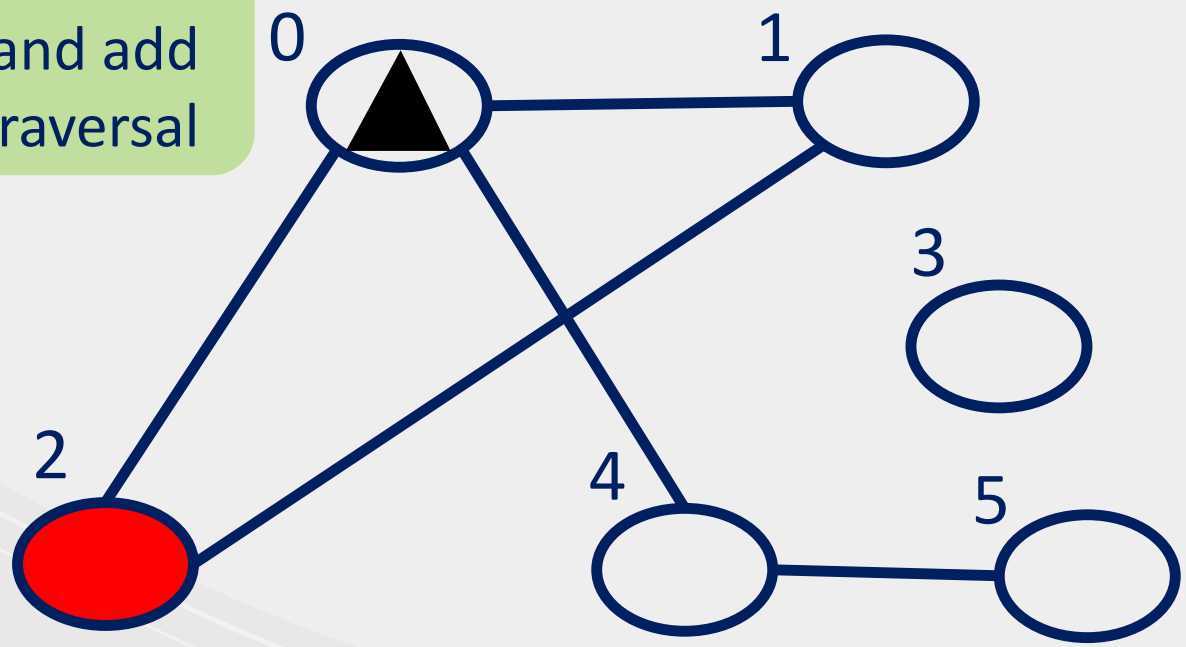
Is the node visited?



Traversal: 2

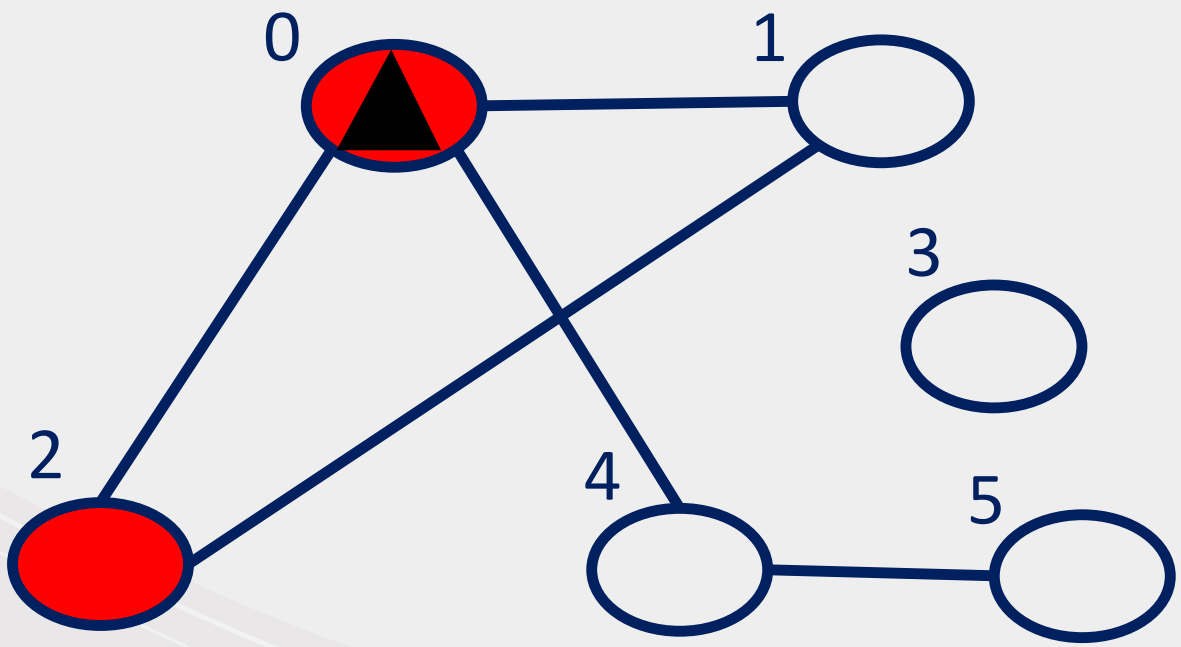
Breadth first search

No! => Make it visited and add to the traversal



Traversal: 2

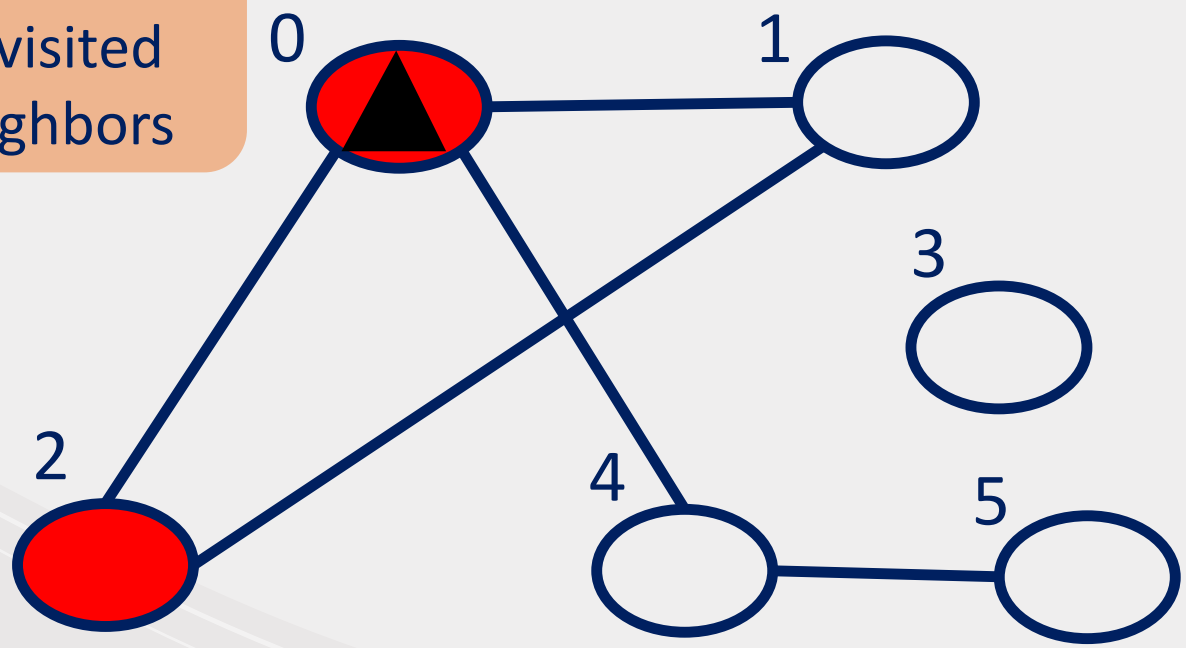
Breadth first search



Traversal: 2,0

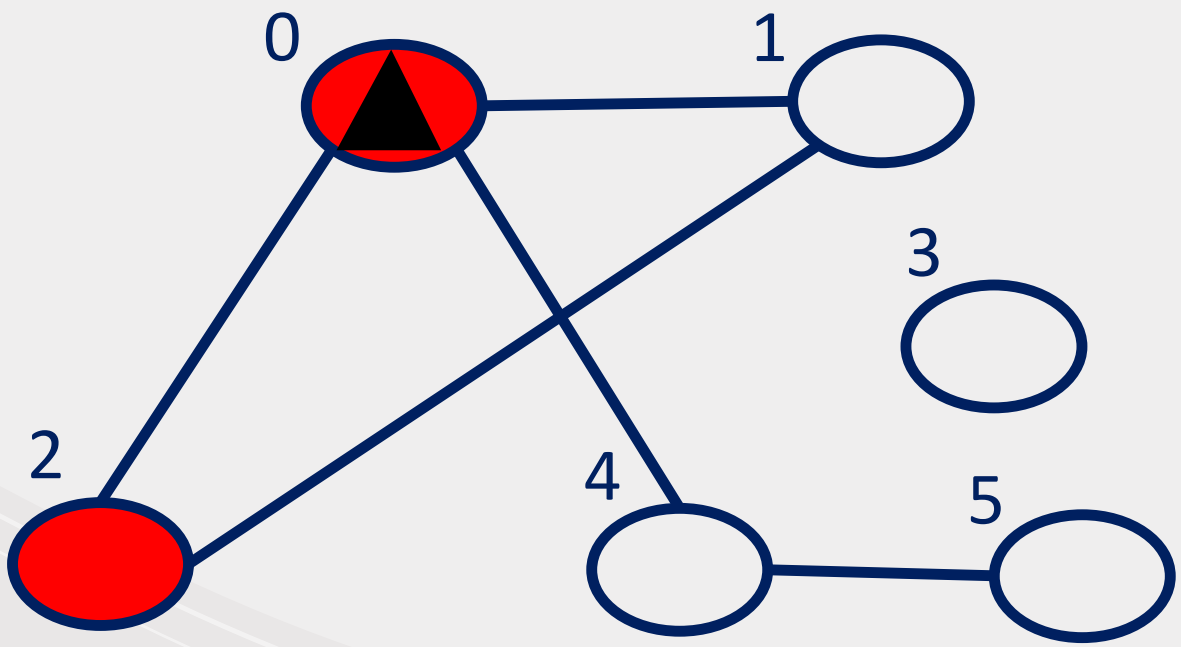
Breadth first search

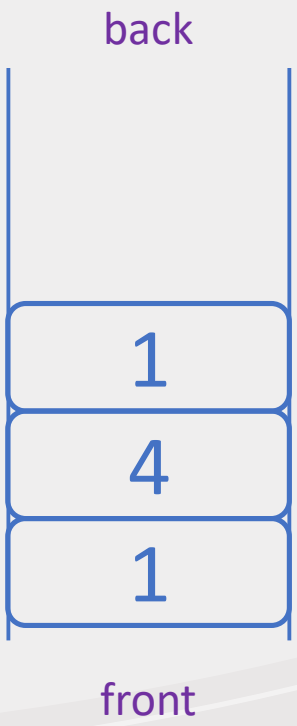
Enqueue all unvisited neighbors



Traversal: 2,0

Breadth first search

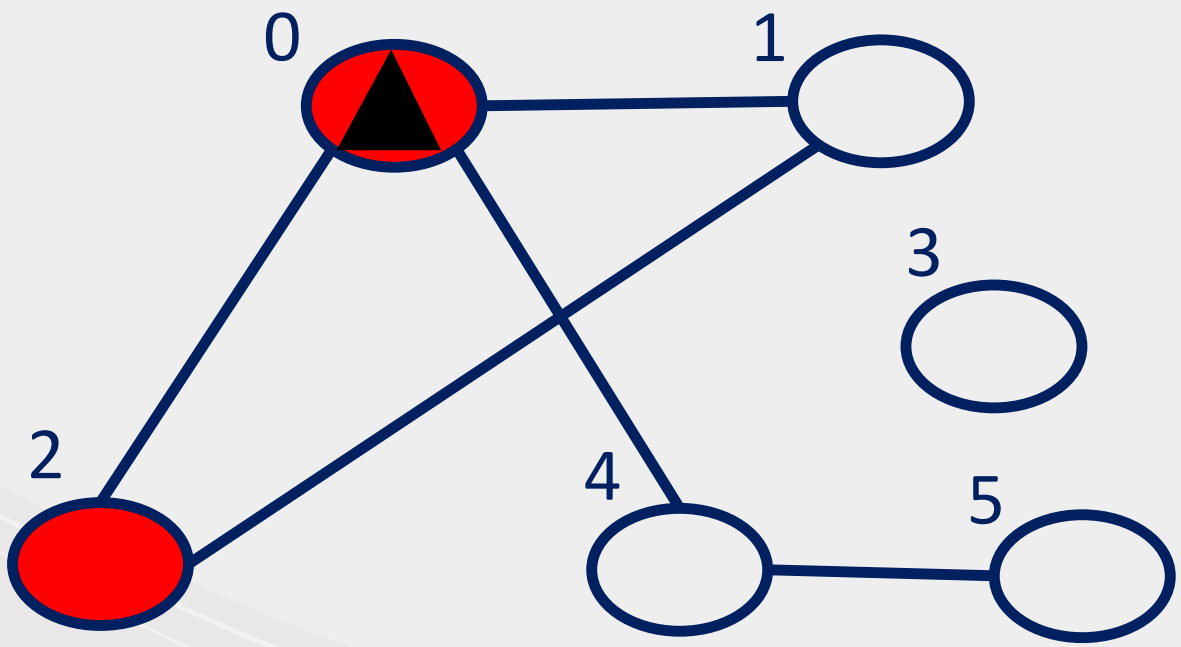


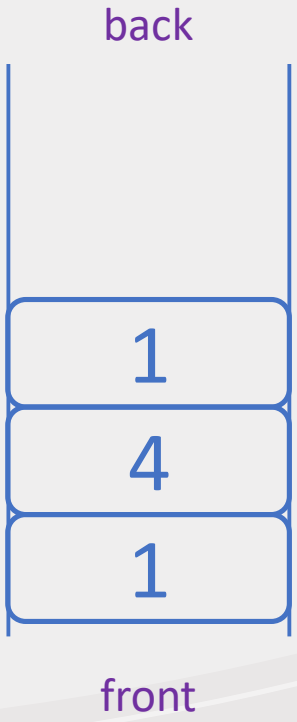


Traversal: 2,0

Breadth first search

Is the queue empty?

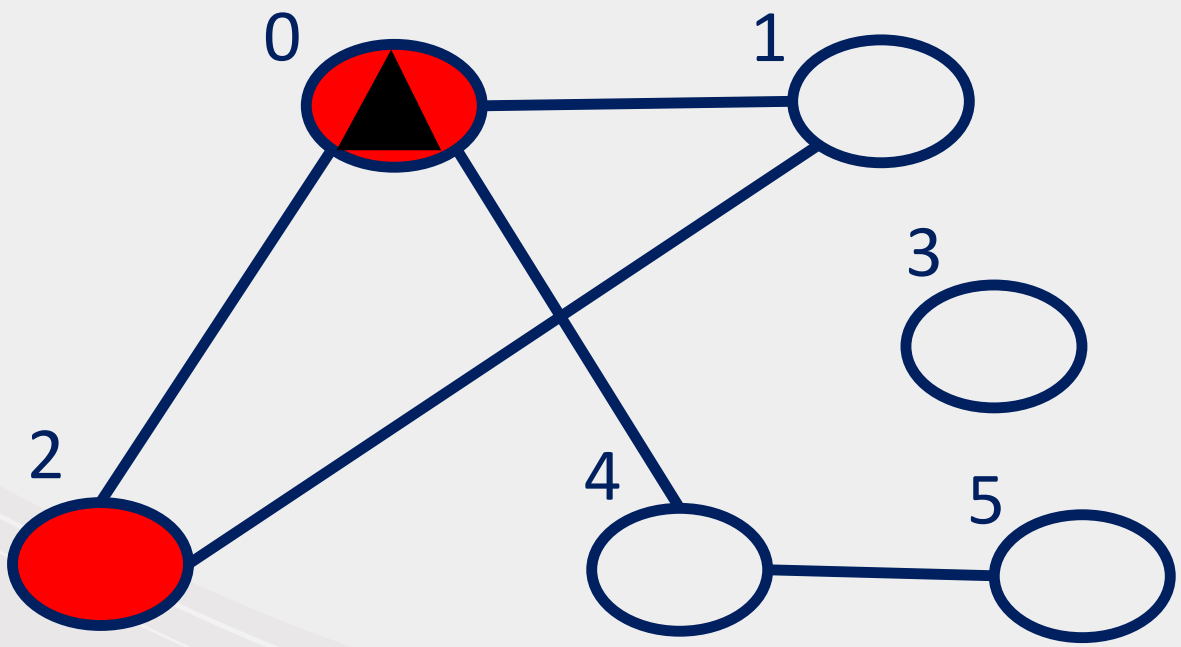


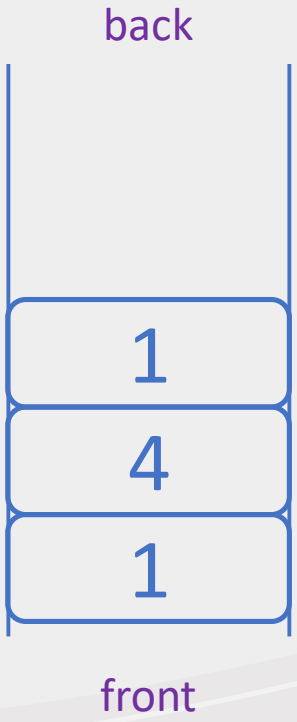


Traversal: 2,0

Breadth first search

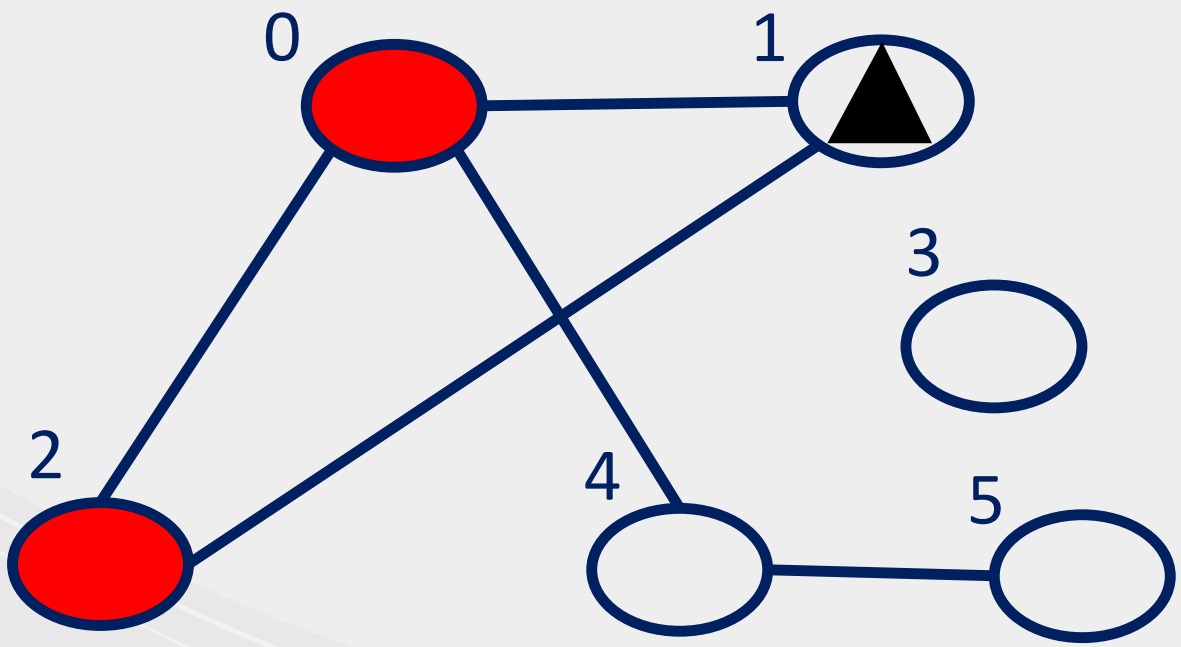
No! => Dequeue the node and make it current

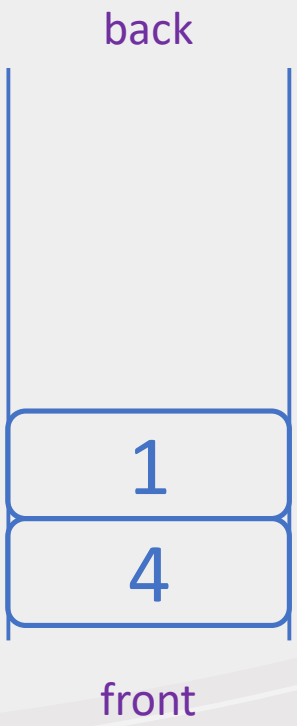




Traversal: 2,0

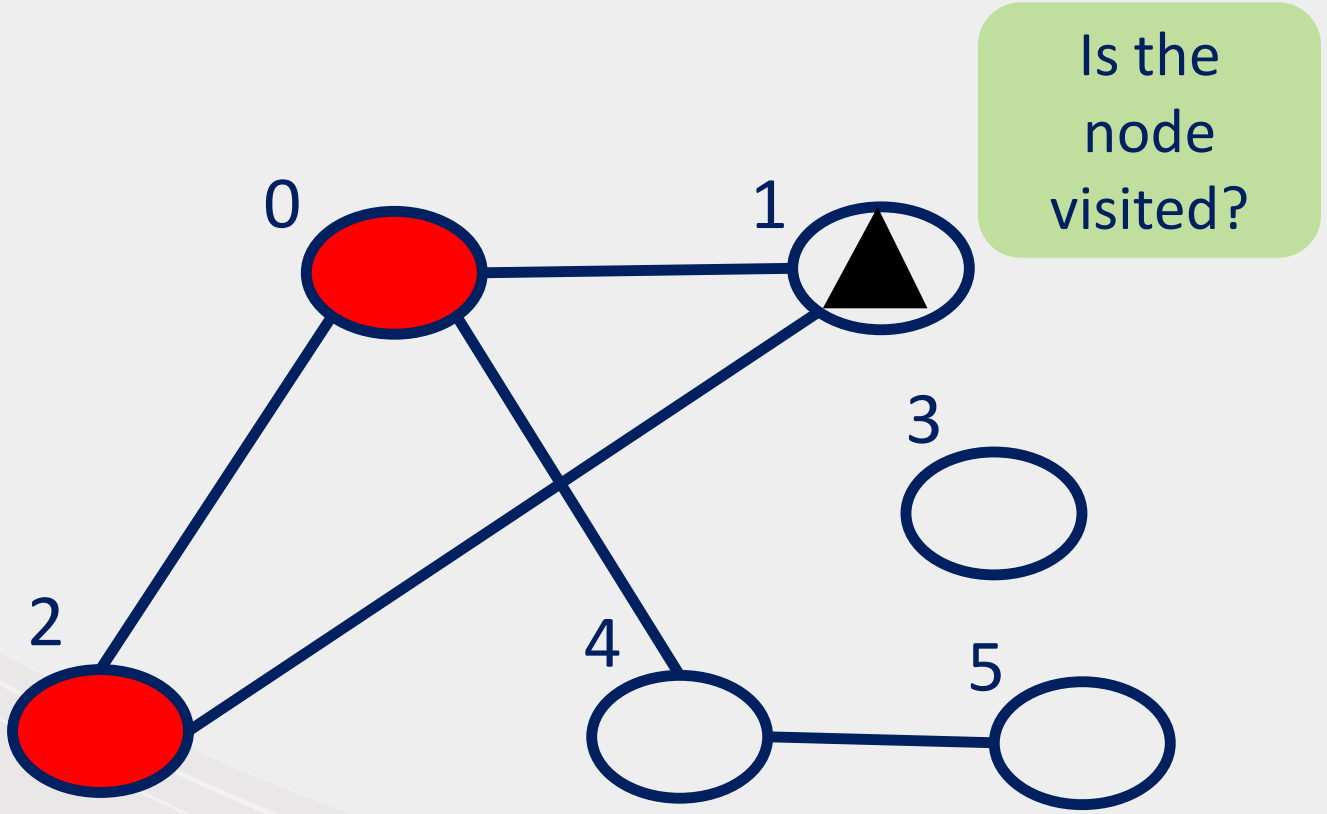
Breadth first search



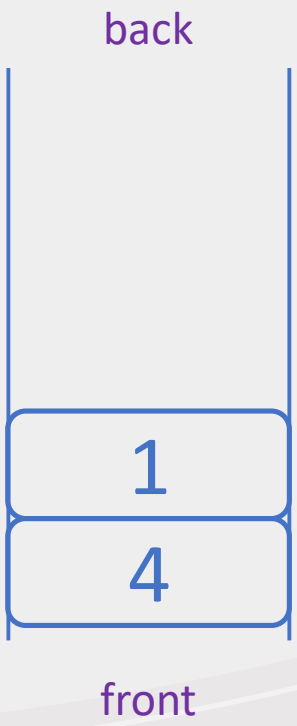


Traversal: 2,0

Breadth first search

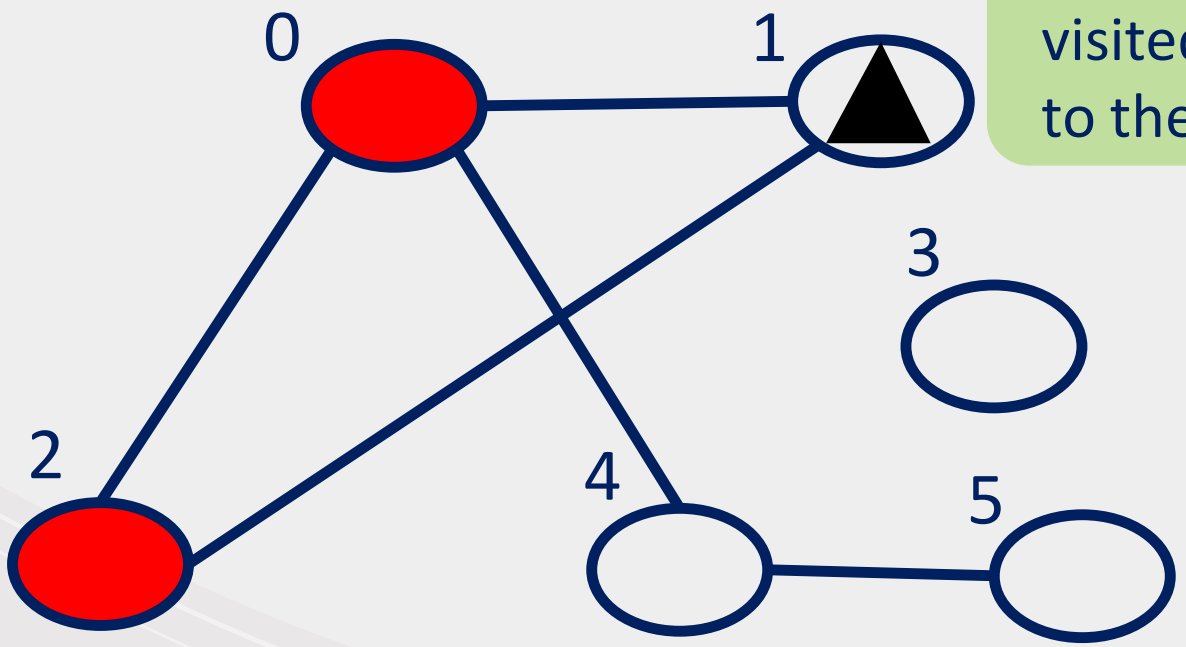


Is the node visited?

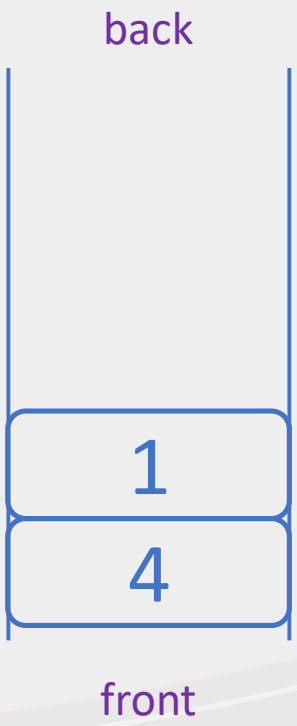


Traversal: 2,0

Breadth first search

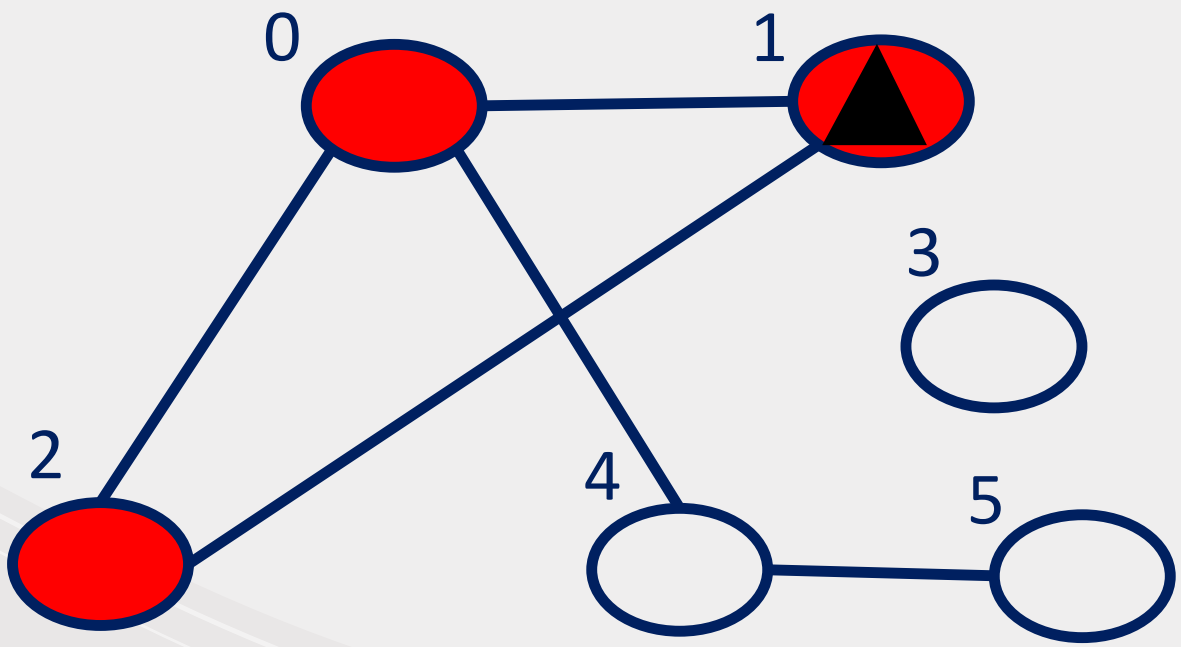


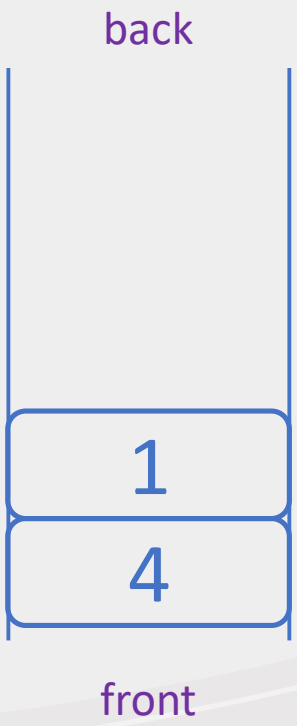
No! => Make it visited and add to the traversal



Traversal: 2,0

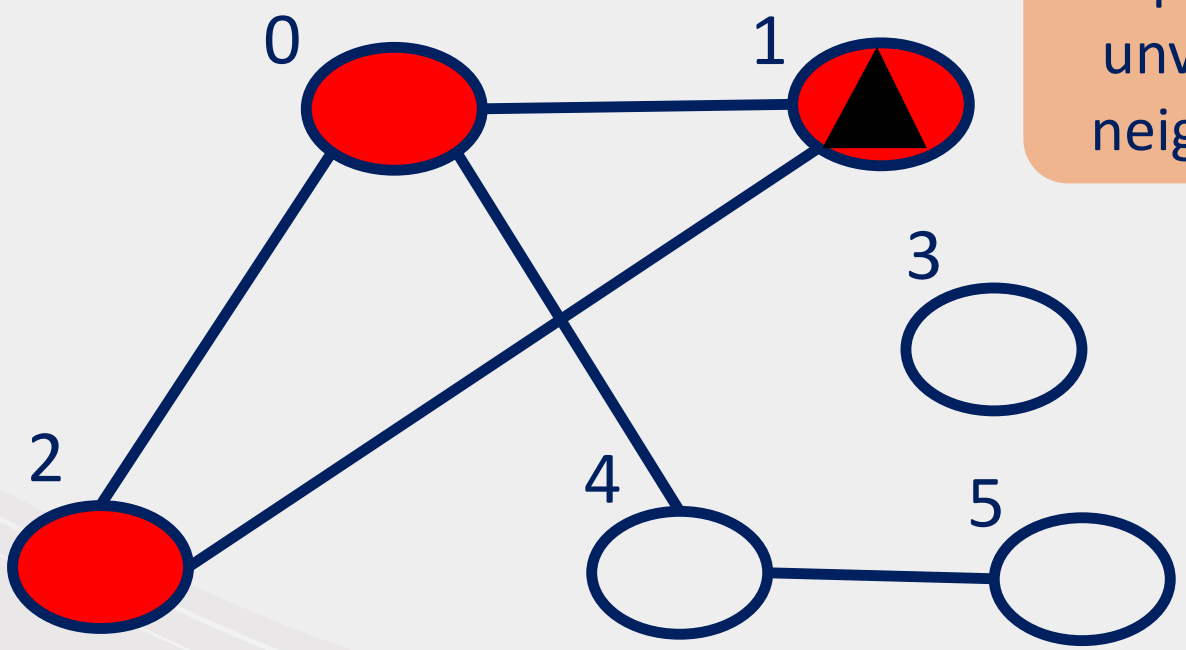
Breadth first search



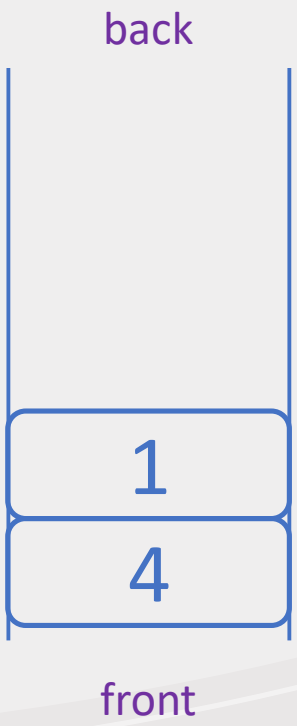


Traversal: 2,0,1

Breadth first search

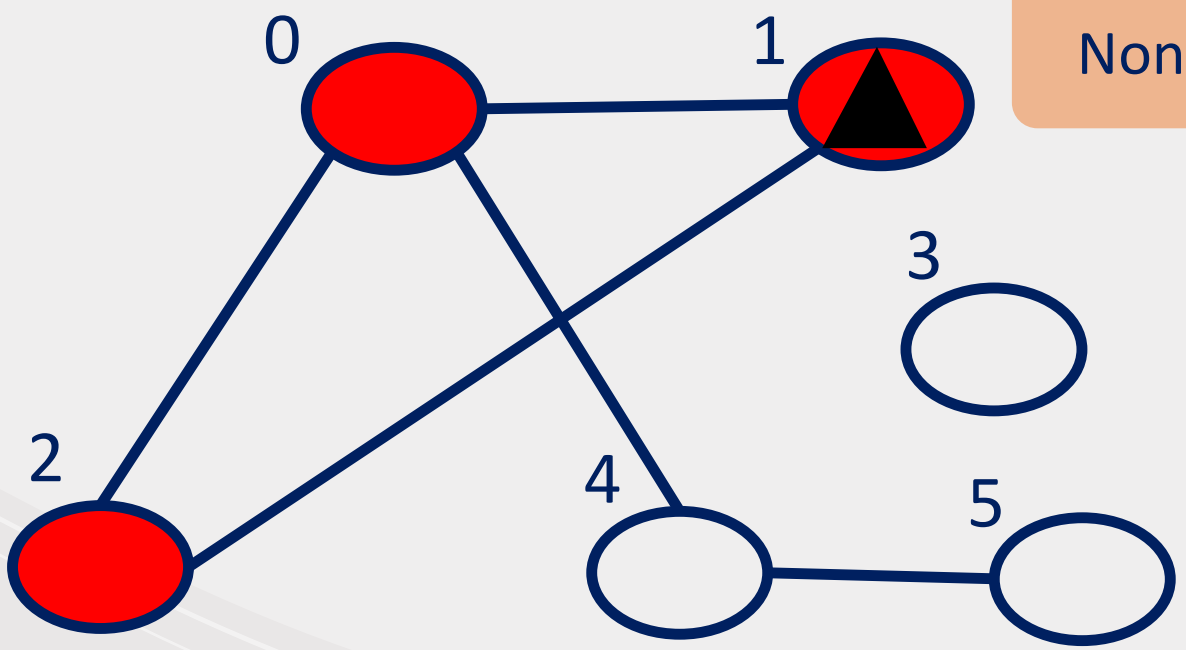


Enqueue all unvisited neighbors

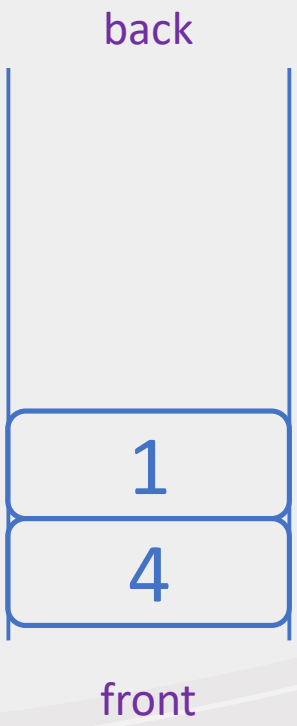


Traversal: 2,0,1

Breadth first search



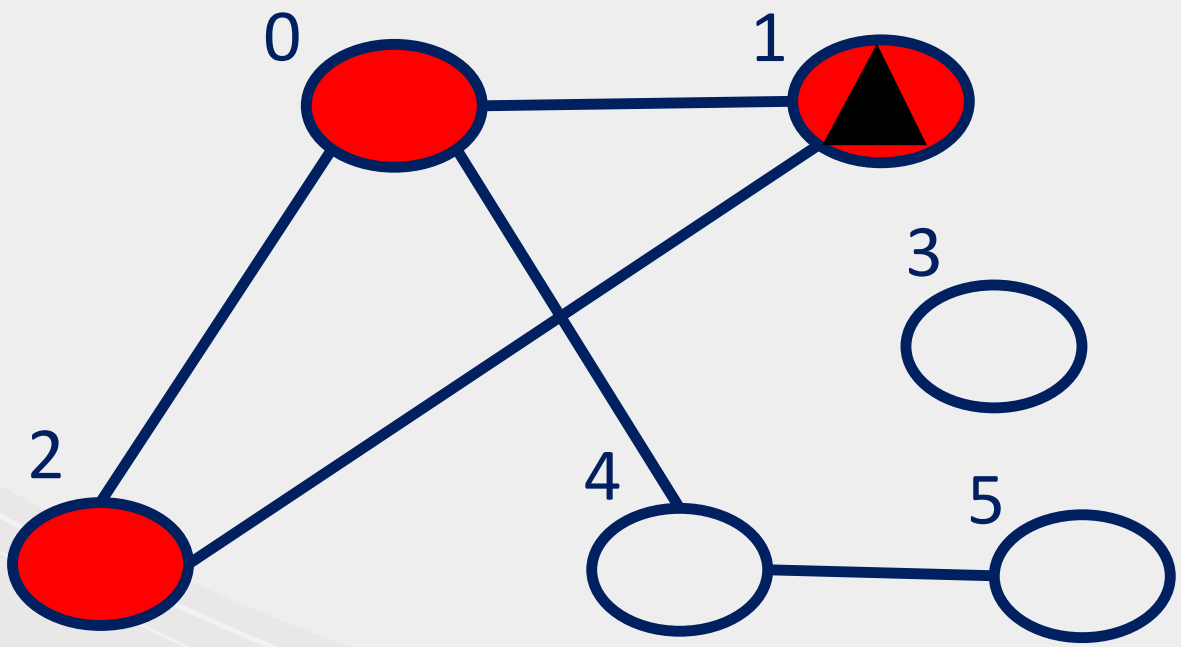
None exist!

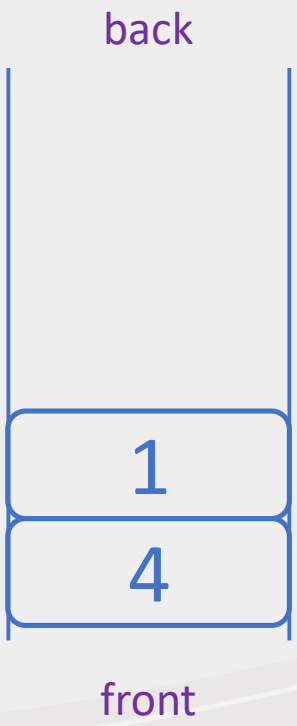


Traversal: 2,0,1

Breadth first search

Is the queue empty?

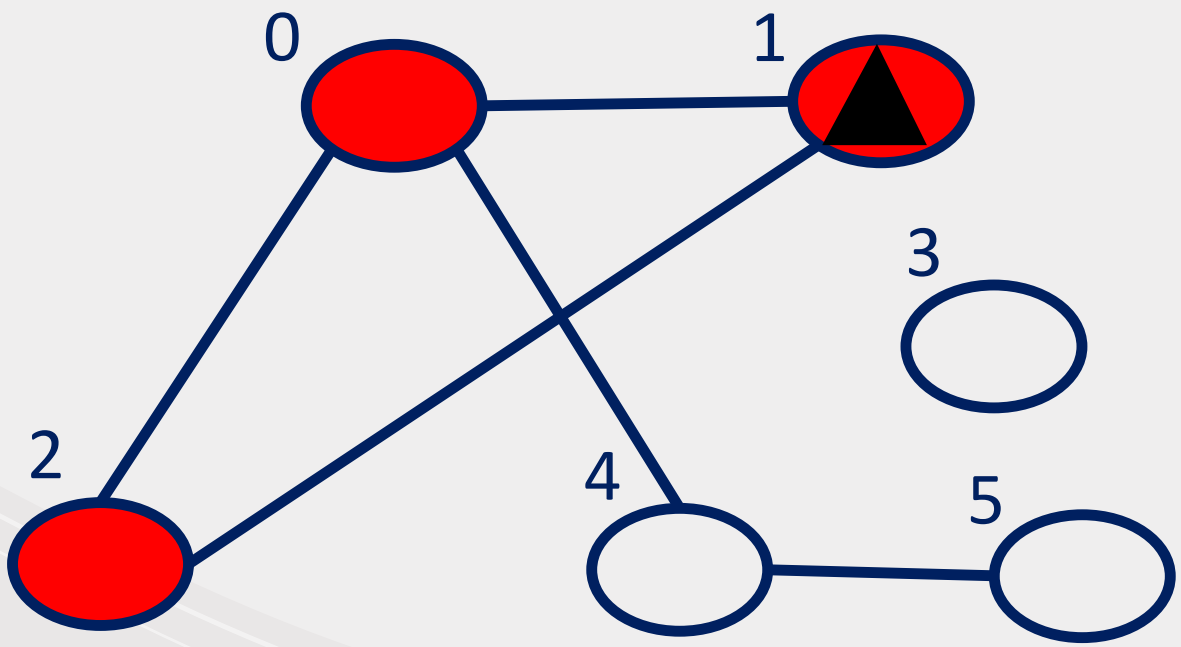


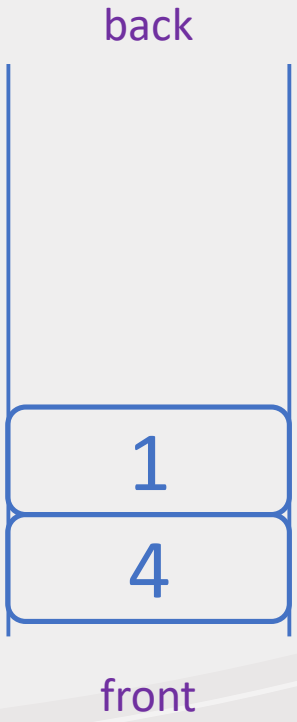


Traversal: 2,0,1

Breadth first search

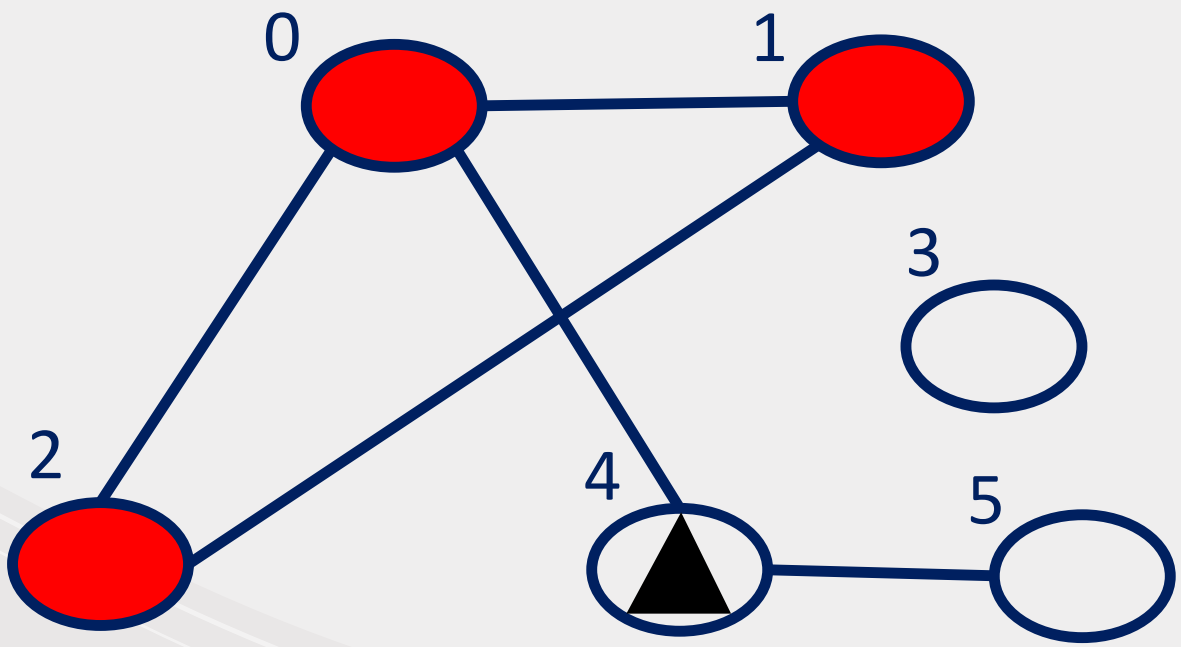
No! => Dequeue the node and make it current





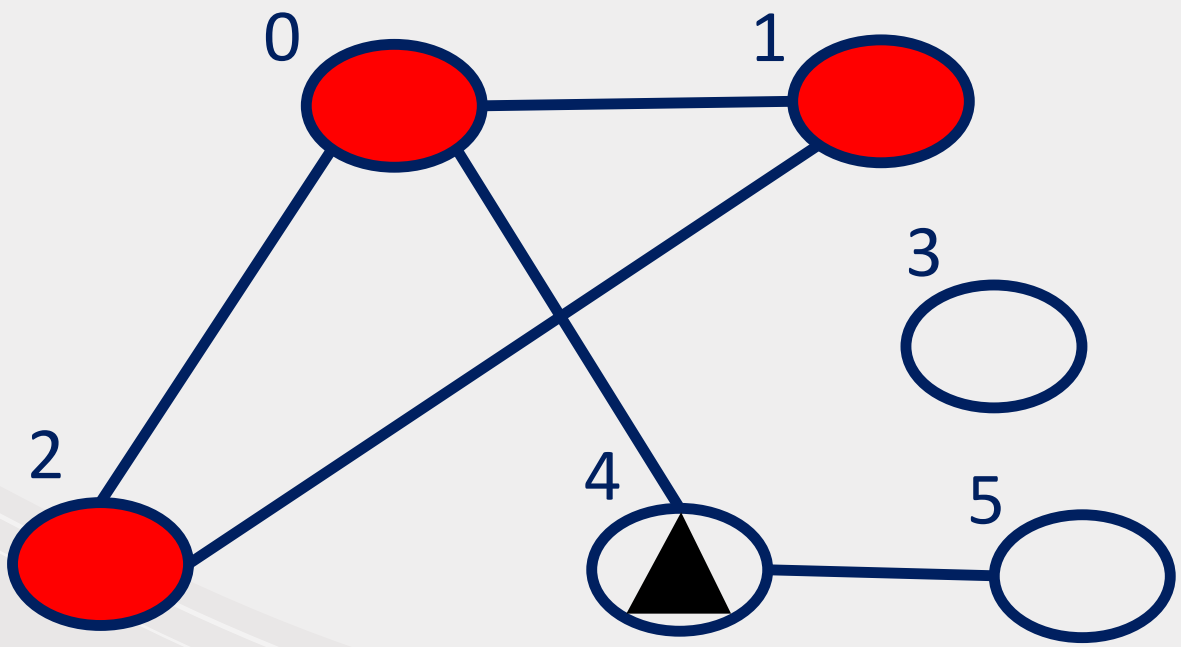
Traversal: 2,0,1

Breadth first search



Traversal: 2,0,1

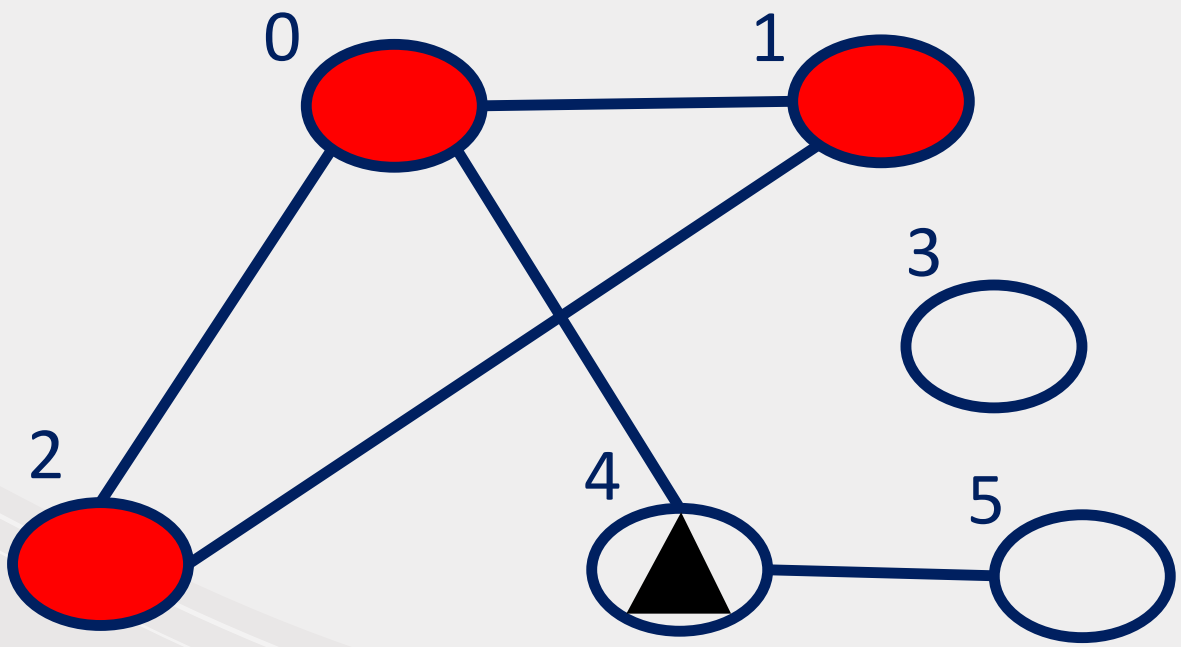
Breadth first search



Is the node visited?

Traversal: 2,0,1

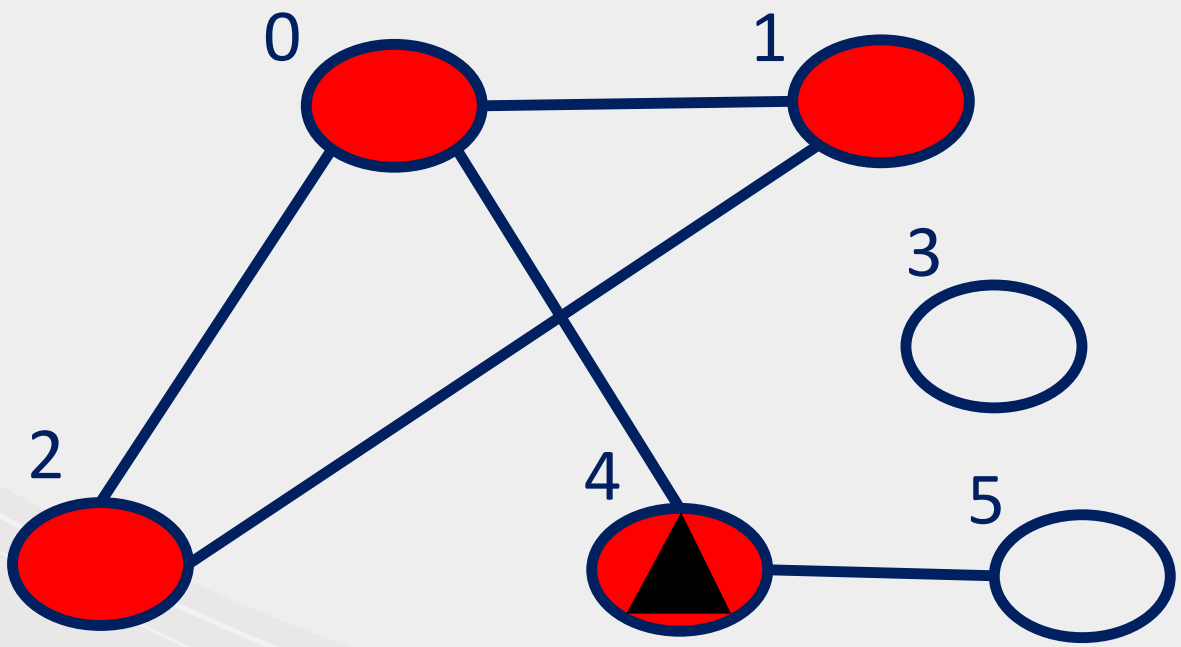
Breadth first search



No! => Make it visited and add to the traversal

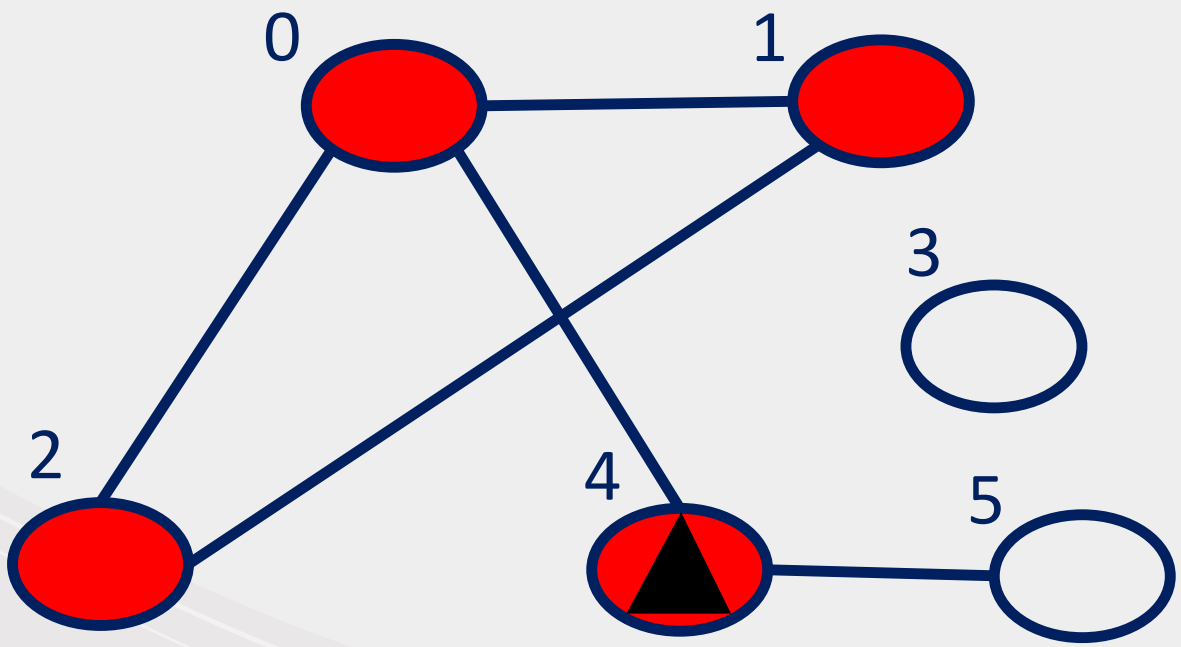
Traversal: 2,0,1

Breadth first search



Traversal: 2,0,1,4

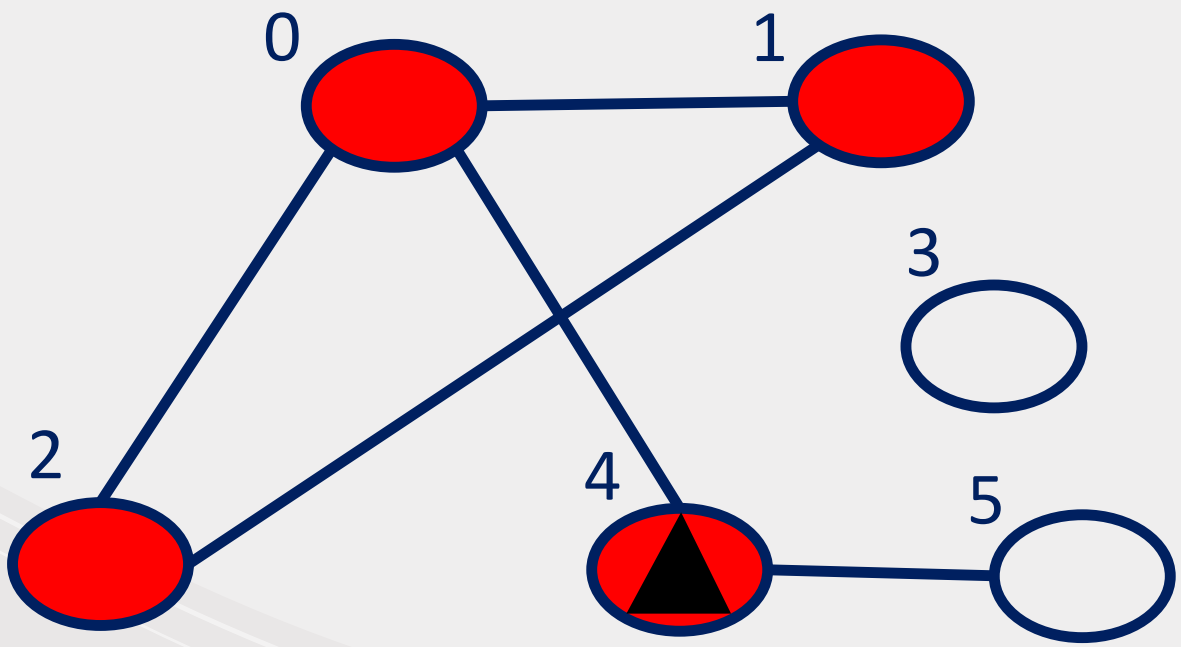
Breadth first search

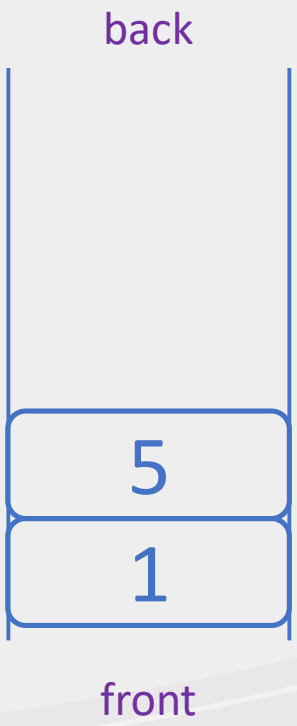


Enqueue all unvisited neighbors

Traversal: 2,0,1,4

Breadth first search

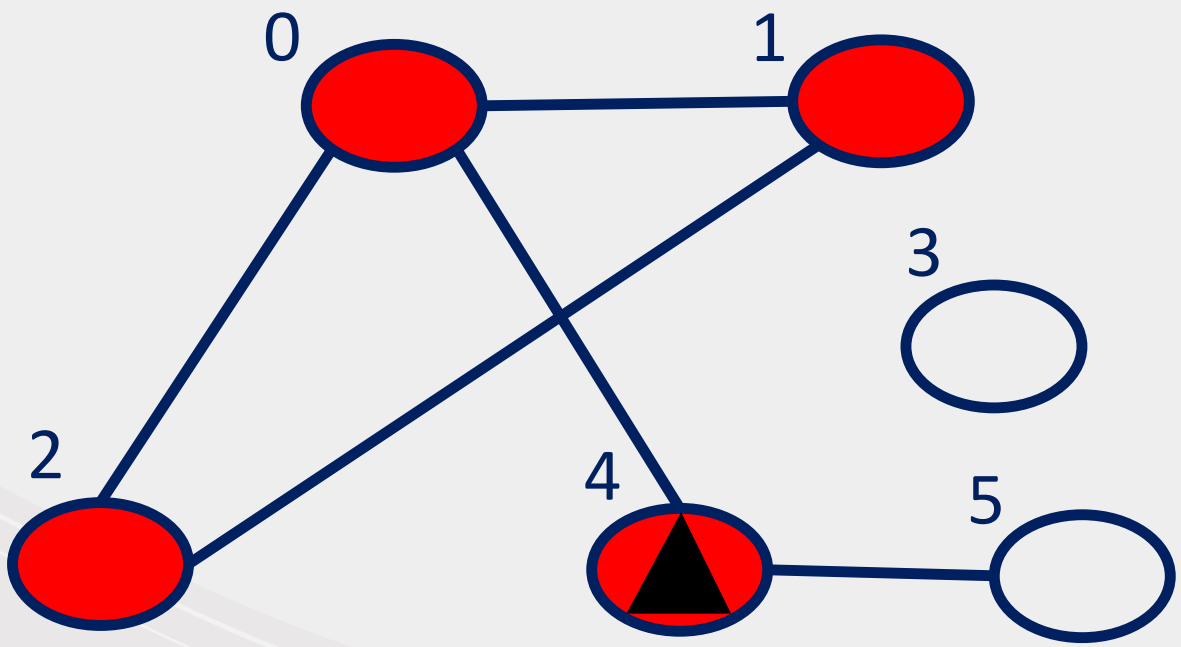


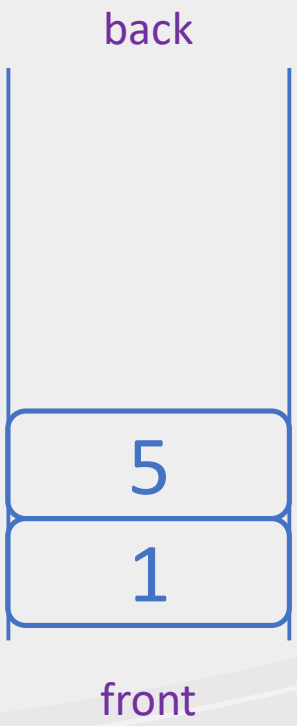


Traversal: 2,0,1,4

Breadth first search

Is the queue empty?

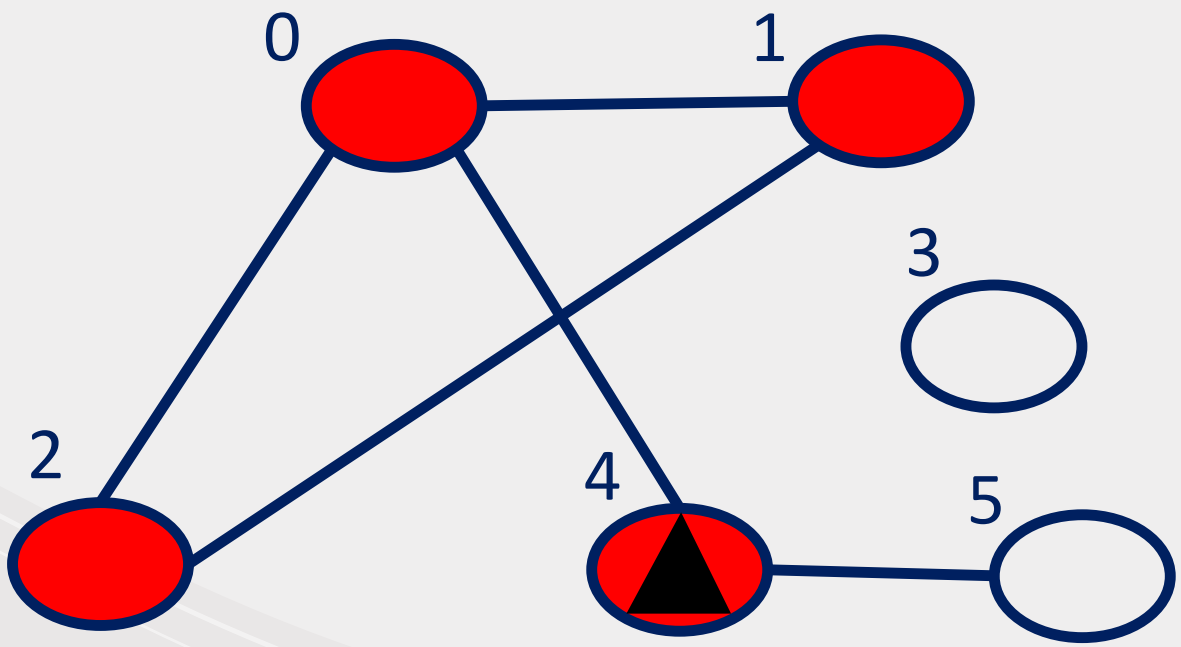


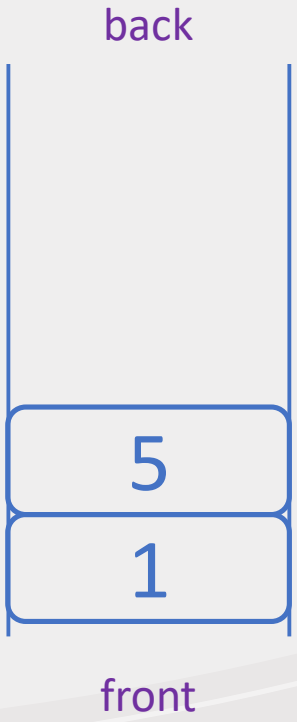


Traversal: 2,0,1,4

Breadth first search

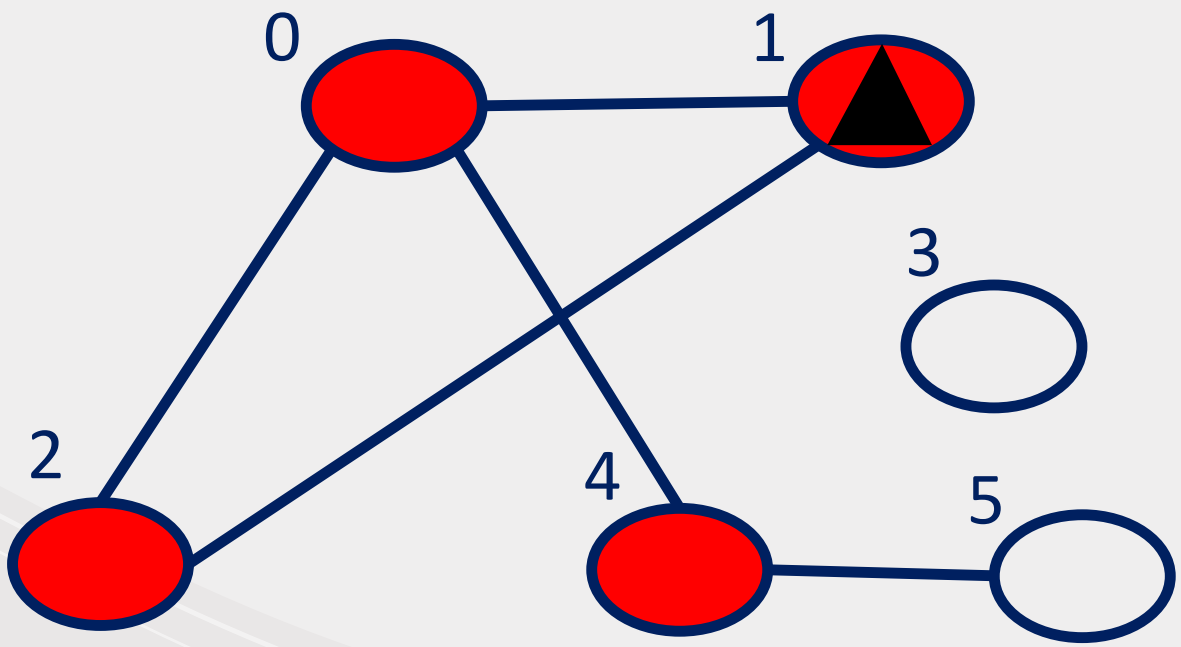
No! => Dequeue the node and make it current





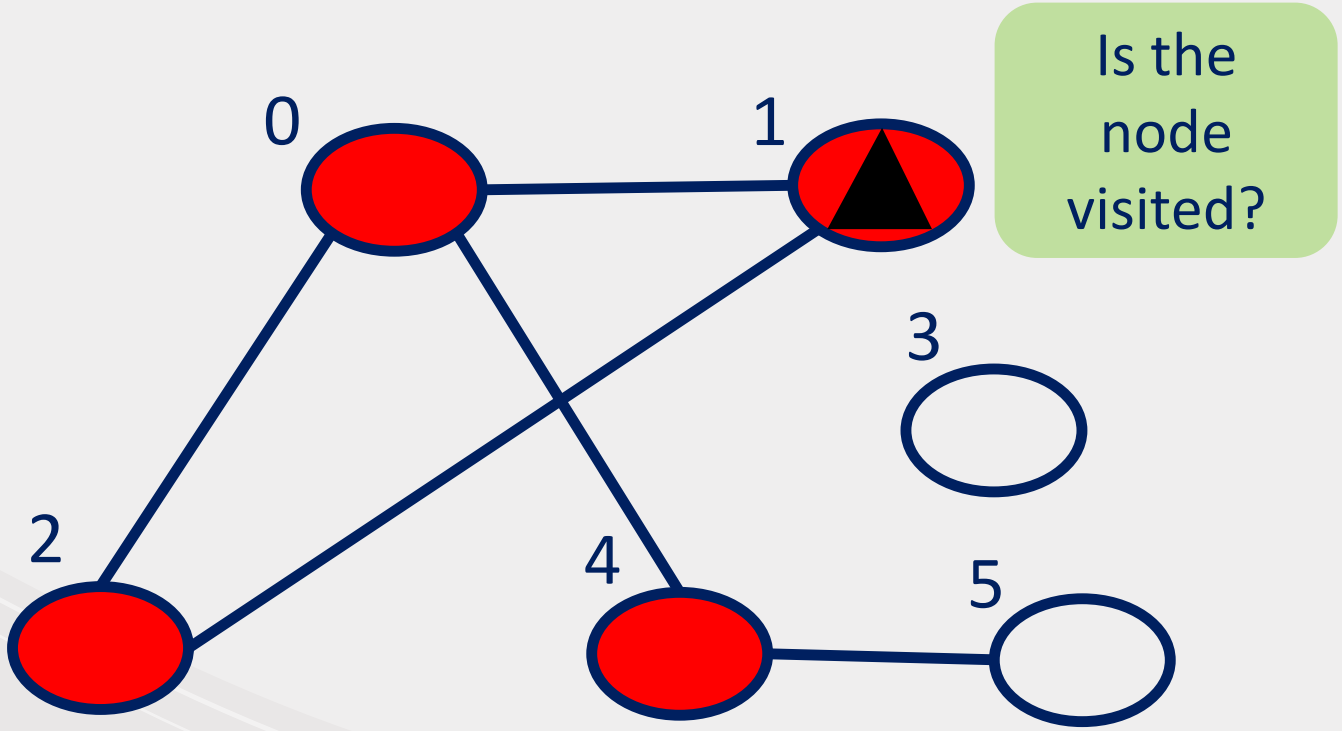
Traversal: 2,0,1,4

Breadth first search



Traversal: 2,0,1,4

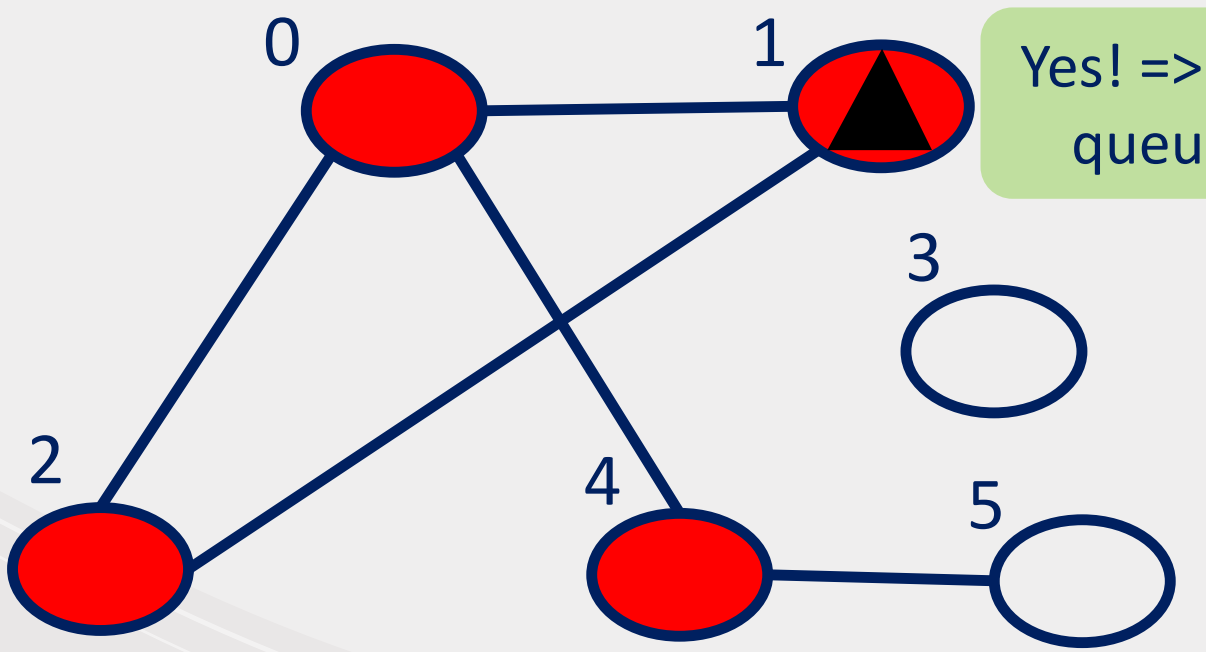
Breadth first search



Is the node visited?

Traversal: 2,0,1,4

Breadth first search

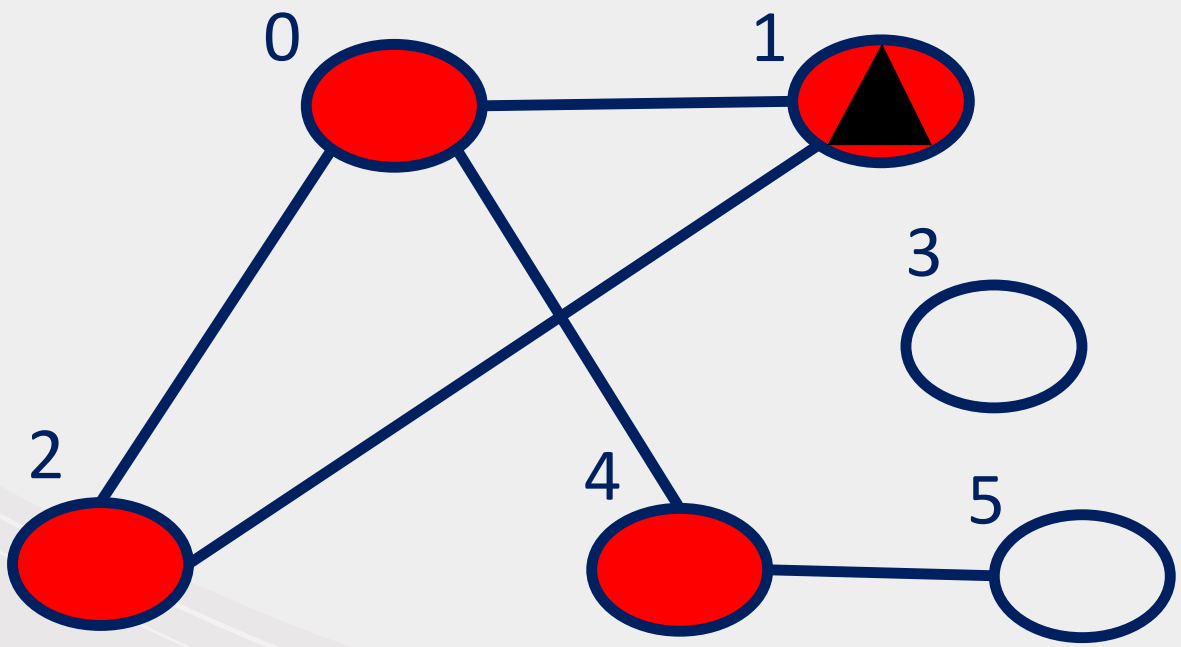


Yes! => Check if the queue is empty

Traversal: 2,0,1,4

Breadth first search

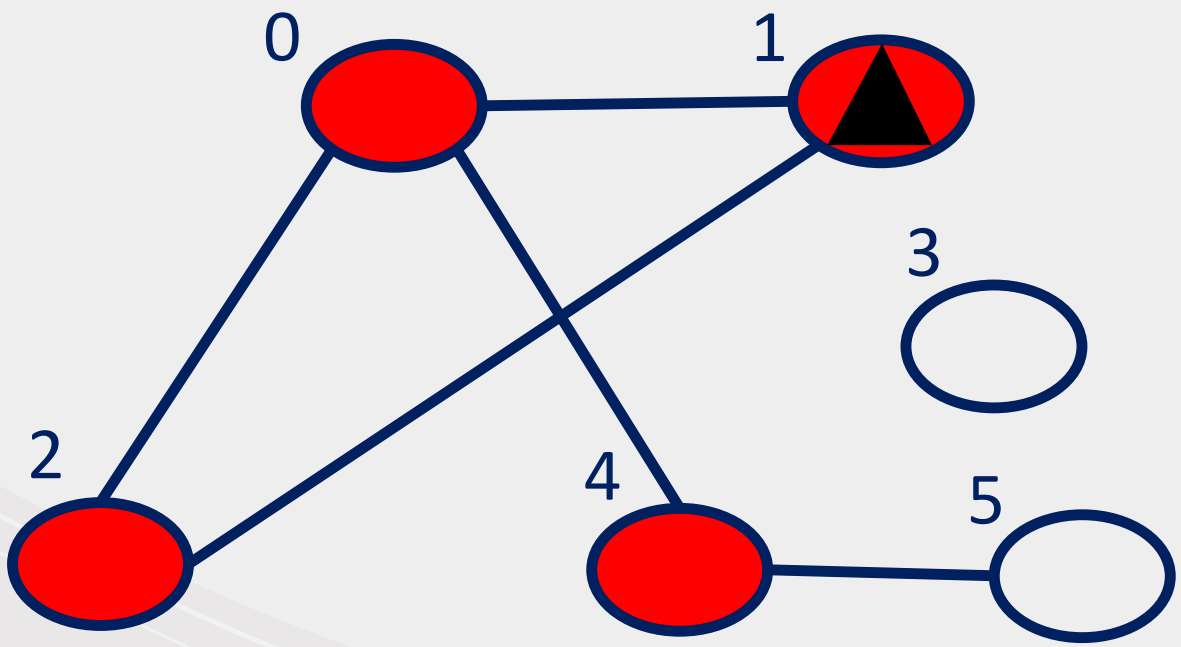
Is the queue empty?



Traversal: 2,0,1,4

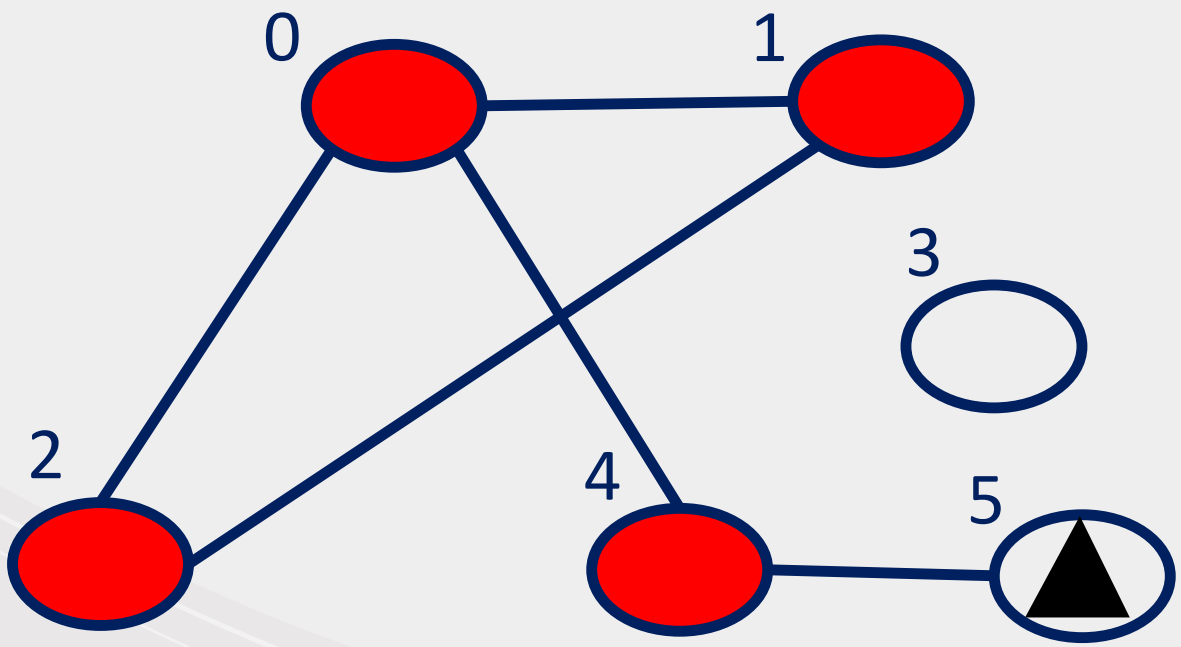
Breadth first search

No! => Dequeue the node and make it current



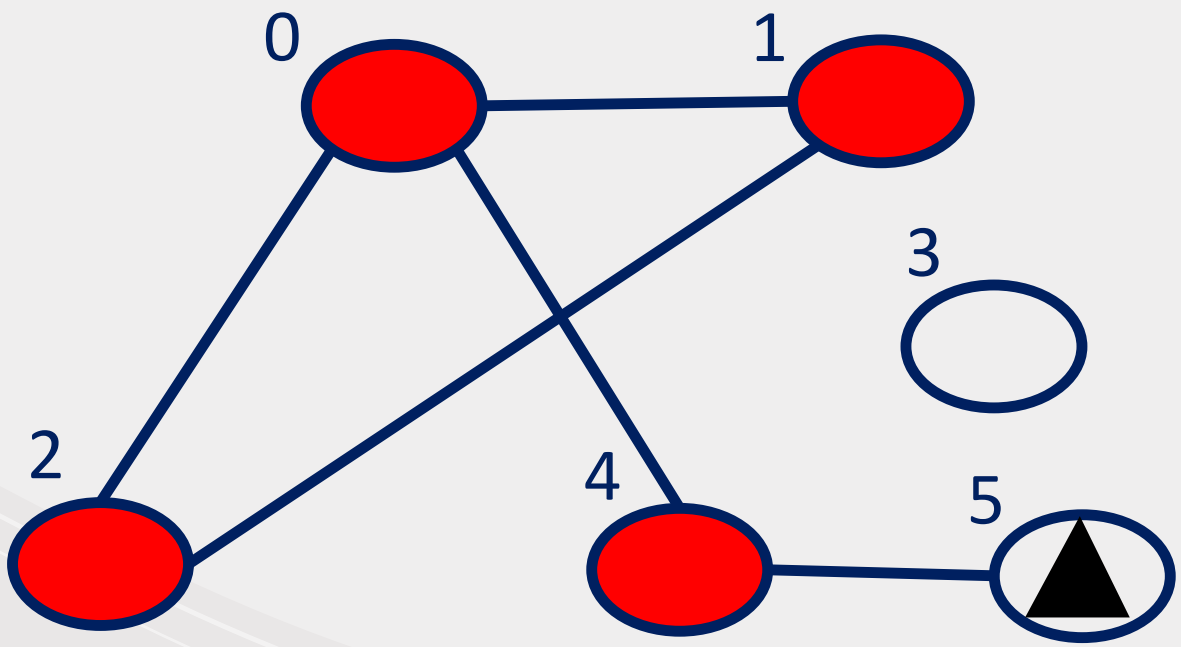
Traversal: 2,0,1,4

Breadth first search



Traversal: 2,0,1,4

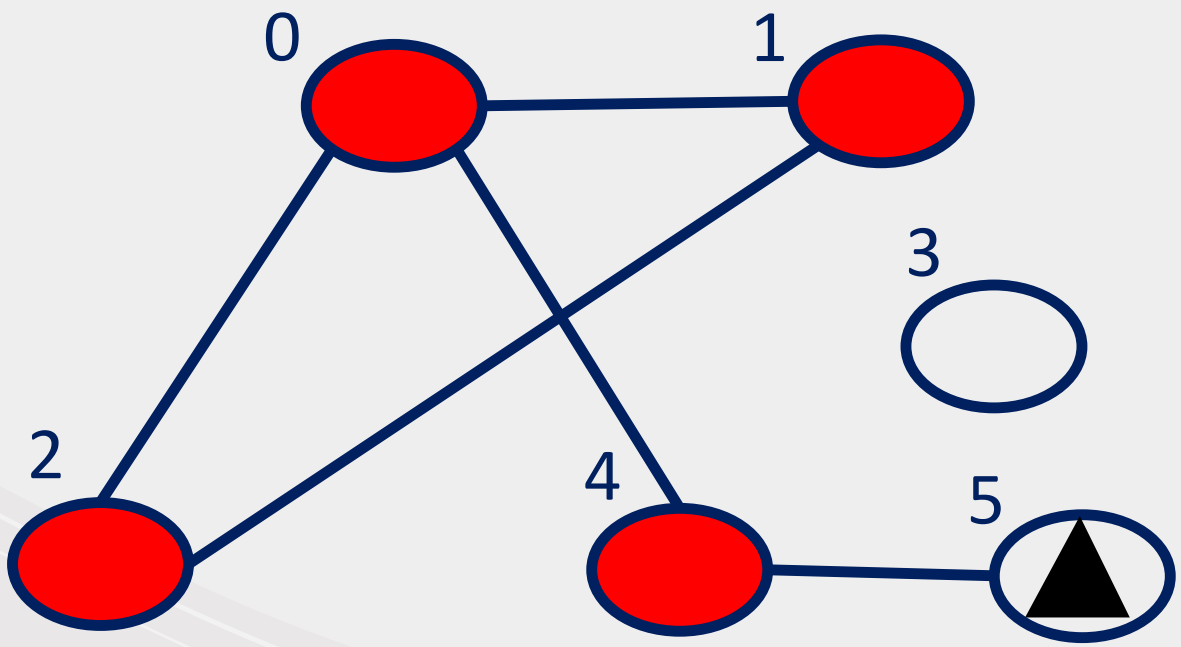
Breadth first search



Is the node visited?

Traversal: 2,0,1,4

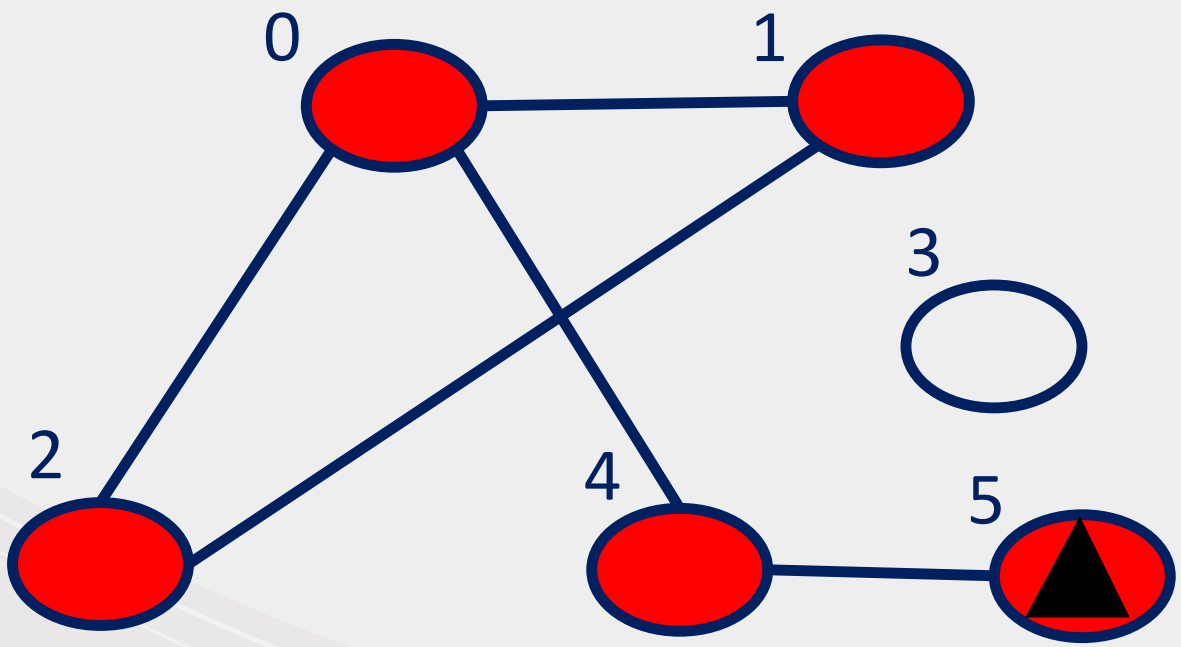
Breadth first search



No! => Make it visited and add to the traversal

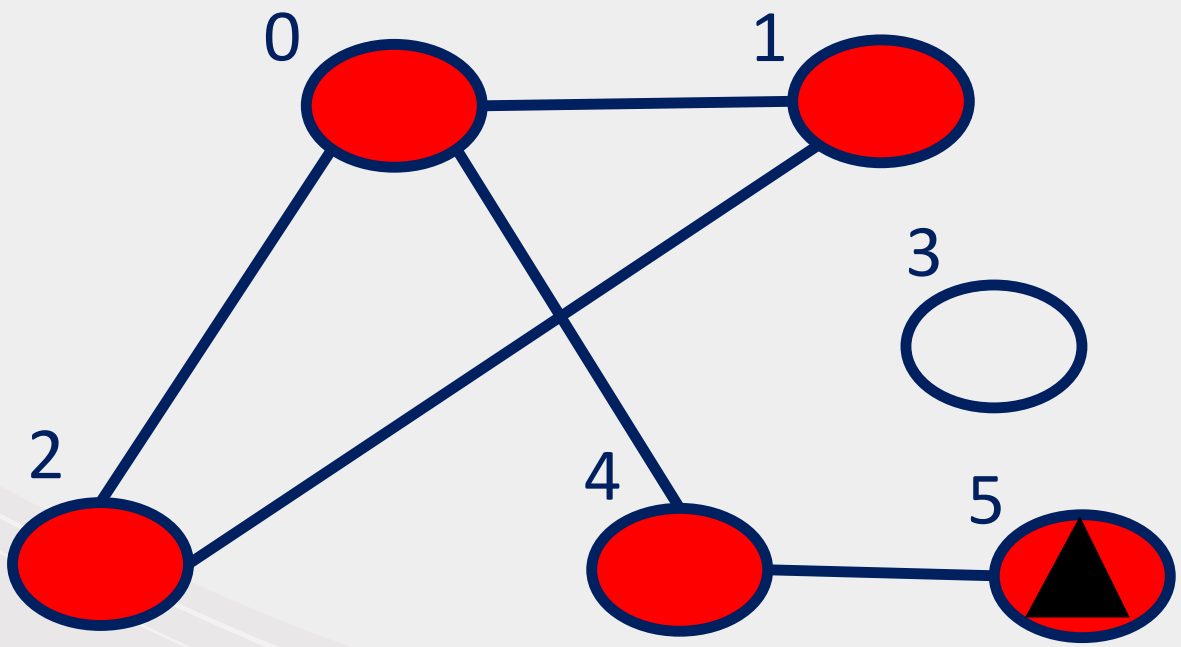
Traversal: 2,0,1,4

Breadth first search



Traversal: 2,0,1,4,5

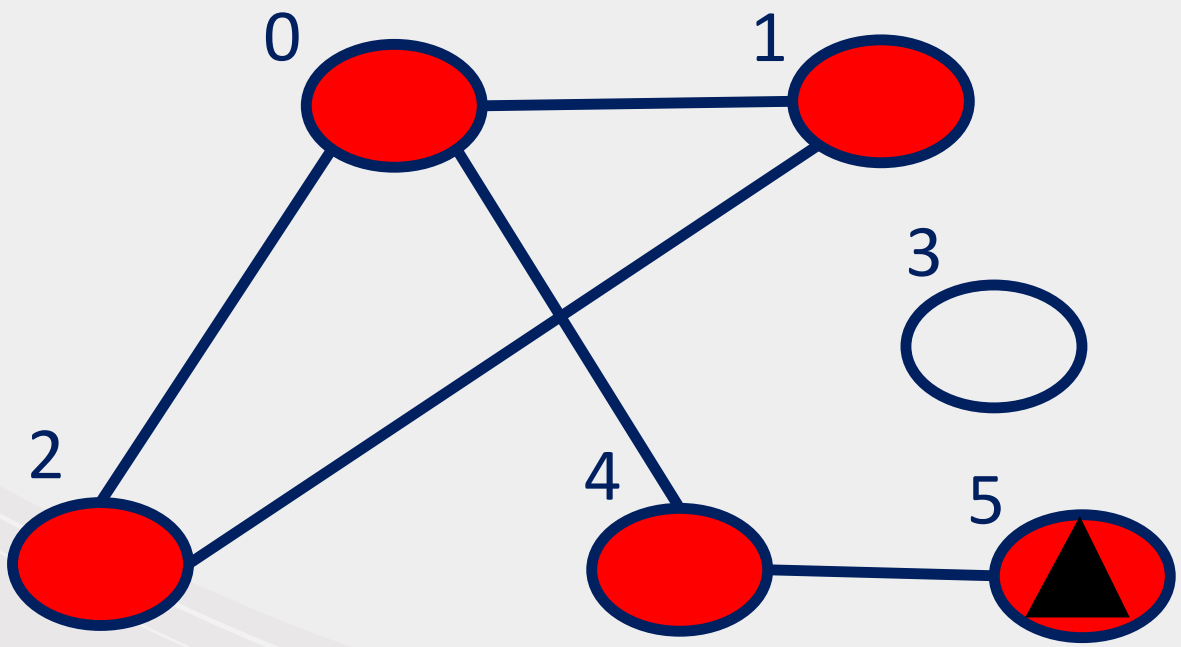
Breadth first search



Enqueue all unvisited neighbors

Traversal: 2,0,1,4,5

Breadth first search

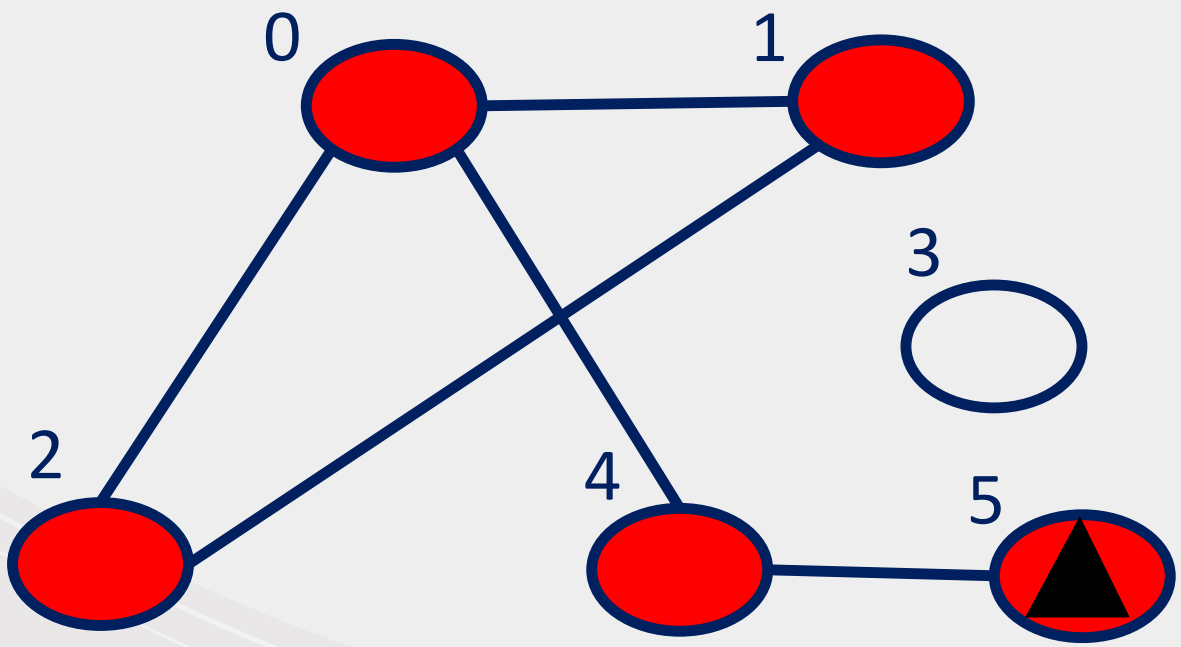


None exist!

Traversal: 2,0,1,4,5

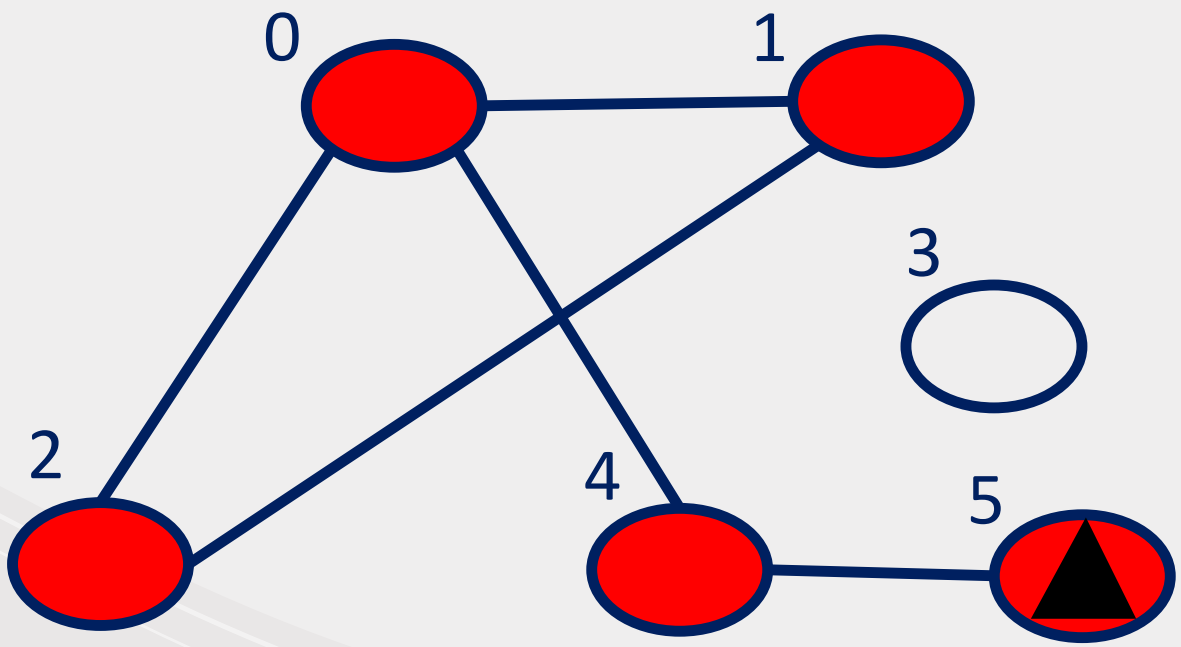
Breadth first search

Is the queue empty?



Traversal: 2,0,1,4,5

Breadth first search



Yes! => STOP

Traversal: 2,0,1,4,5

```
def bfs(self, start):
    traversal = []
    visited = set()  # Set of visited nodes
    queue = deque()  # Queue for BFS
    queue.append(start)

    while queue:
        current = queue.popleft()  # Dequeue the front node
        if current not in visited:
            visited.add(current)
            traversal.append(current)
            for neighbor in self.adj[current]:
                if neighbor not in visited:
                    queue.append(neighbor)  # Enqueue unvisited neighbors
    print(traversal)
```

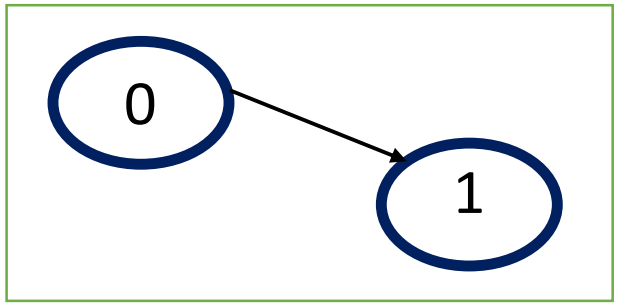
Graph Traversals and their usage

Topological Sorting

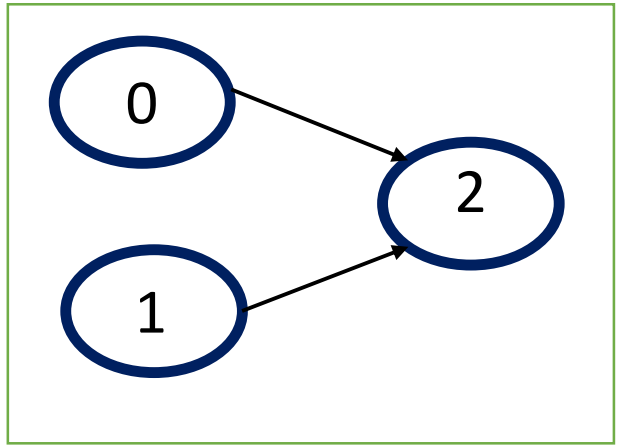
Topological sort

- *Topological* in computer science means pertaining the *network topology*, in our case, the order of nodes
- Takes as an input a *directed* ***acyclic*** graph only
- Returns a list of nodes
- Each node in the output appears before all the nodes it points to
- Multiple sortings are possible

Topological sort – Simple Examples

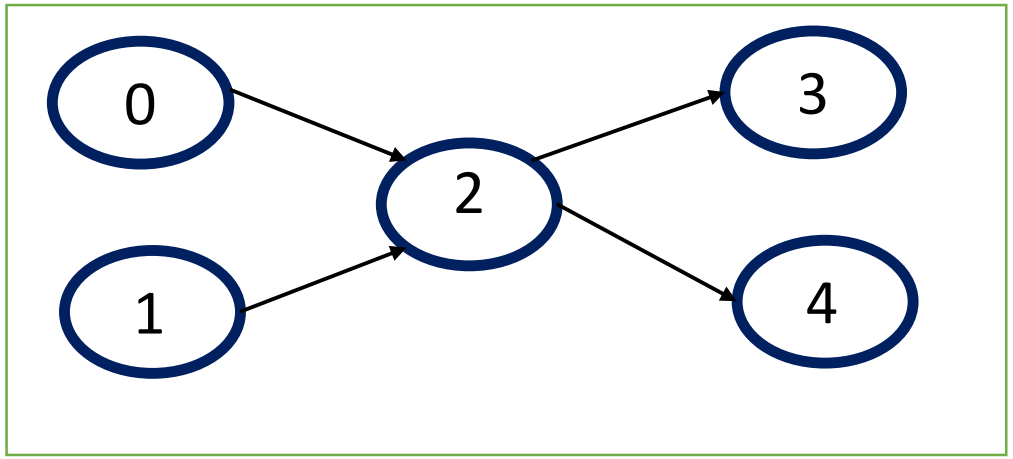


res1: 0, 1



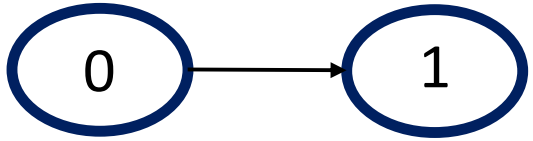
res1: 0, 1, 2
res2: 1, 0, 2

res1: 0, 1, 2, 3, 4
res2: 0, 1, 2, 4, 3
res3: 1, 0, 2, 3, 4
res4: 1, 0, 2, 4, 3

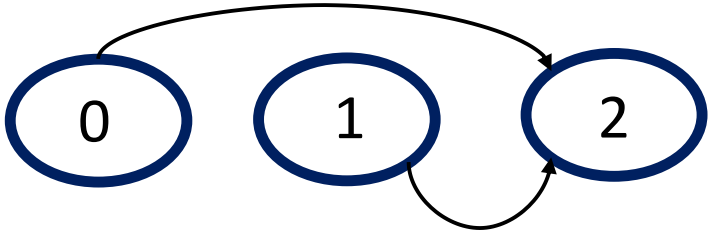


Note: Each node in the output appears before all the nodes it points to

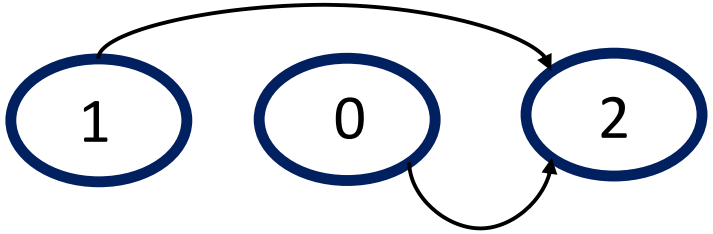
Topological sort – Simple Examples



res1: 0, 1

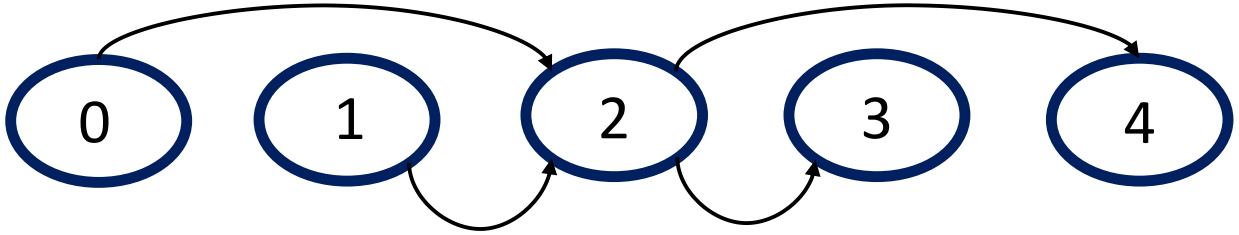


res1: 0, 1, 2

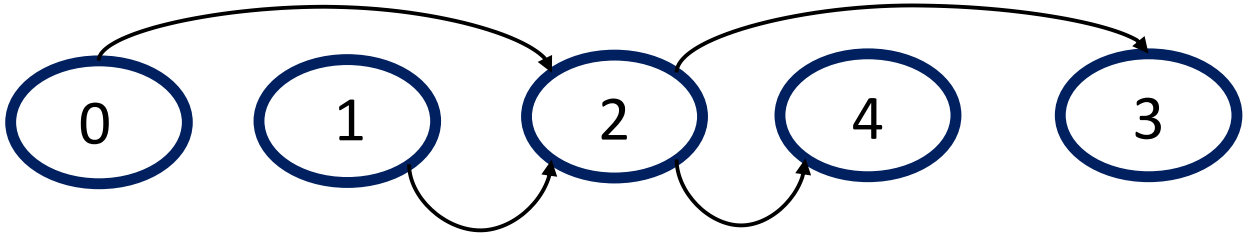


res2: 1, 0, 2

res1: 0, 1, 2, 3, 4

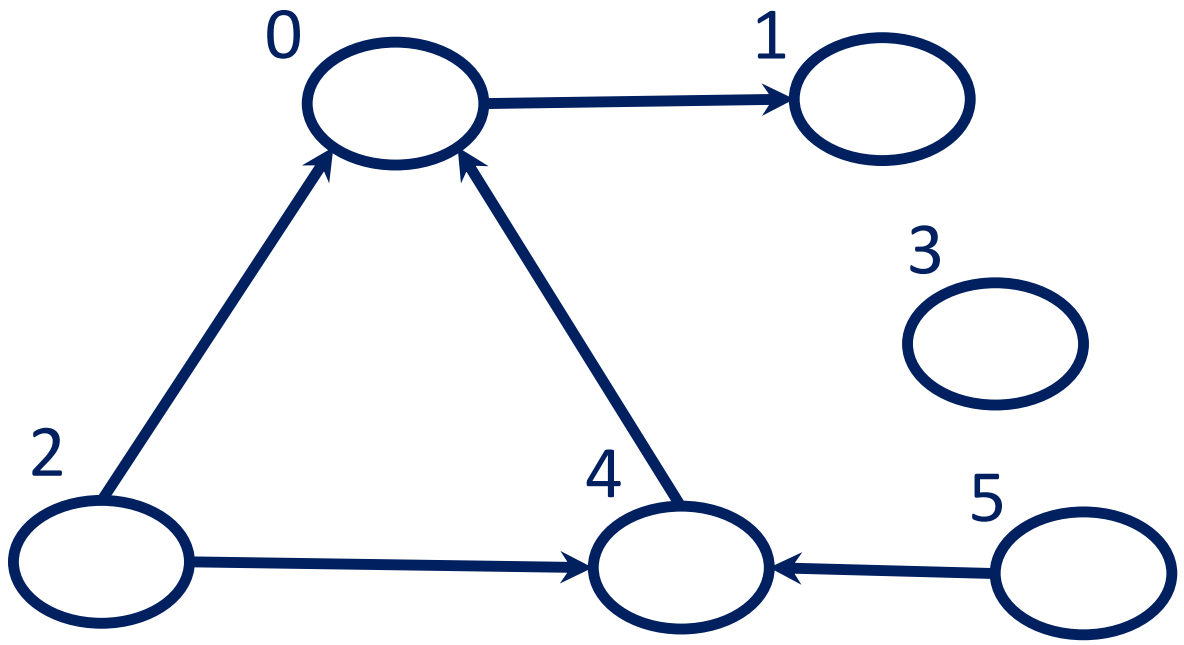


res2: 0, 1, 2, 4, 3



…

Topological sort - examples



- Several sorting are possible:
  - ✓ 5,2,4,0,1,3
  - ✓ 2,5,4,0,1,3
  - ✓ 5,2,4,0,3,1
  - ✓ 2,5,4,0,3,1
  - ✓ ... many other ways since 3 may be anywhere

Topological sort – DFS Based Solution

DFS high-level Steps:
Step 1: Start processing a node (START TIME)
Step 2: Explore all unvisited neighbors (takes some time)
Step 3: Finish processing a node (FINISH TIME)

IDEA:

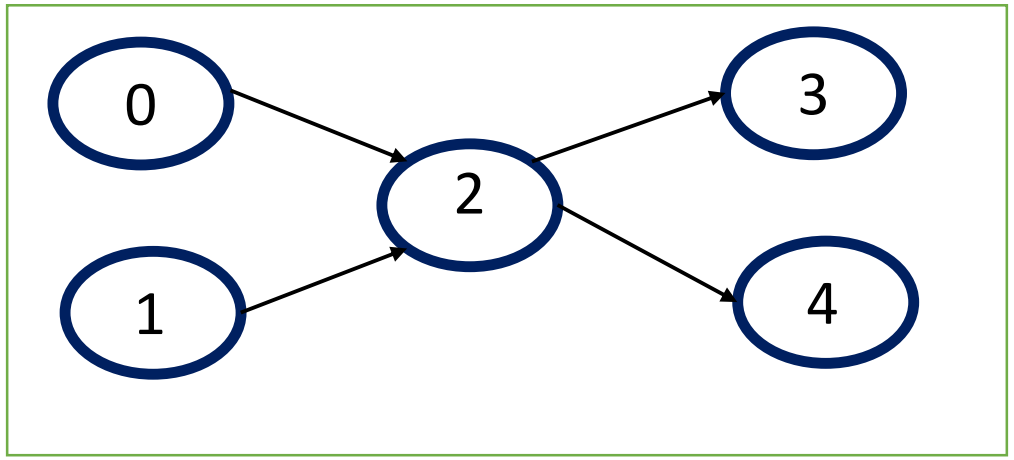
- While performing DFS traversal, compute the **FINISH TIME** for each node.
- Sort the nodes based on the descending order of their FINISH TIMEs

Computing Finish Time - Exercises

For each `node` in the graph:

If `node` is not visited, call **dfs(node)**

dfs(0)

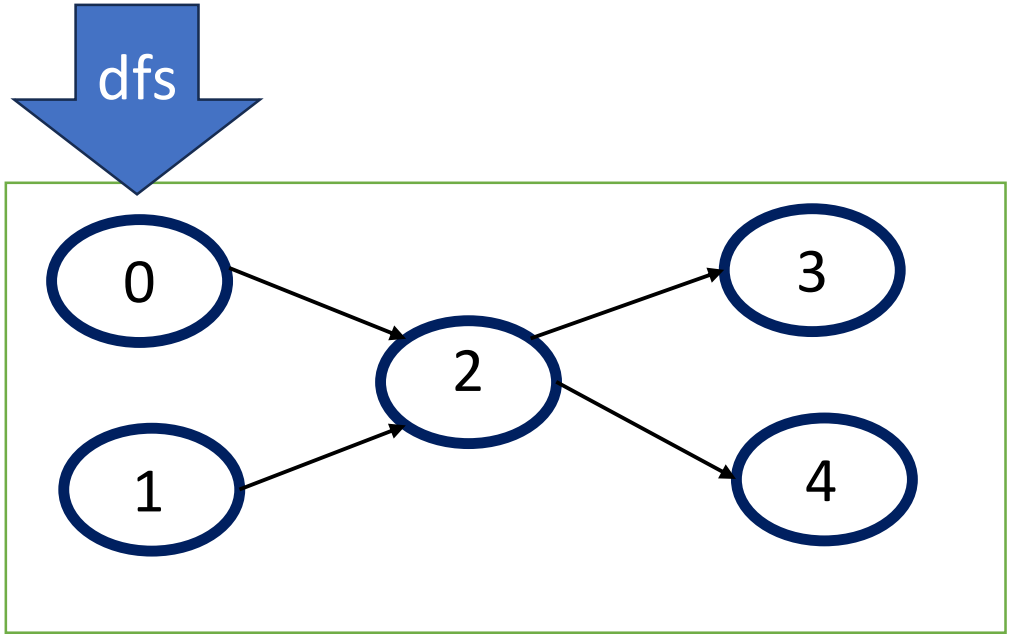


Computing Finish Time - Exercises

For each `node` in the graph:

If `node` is not visited, call **dfs(node)**

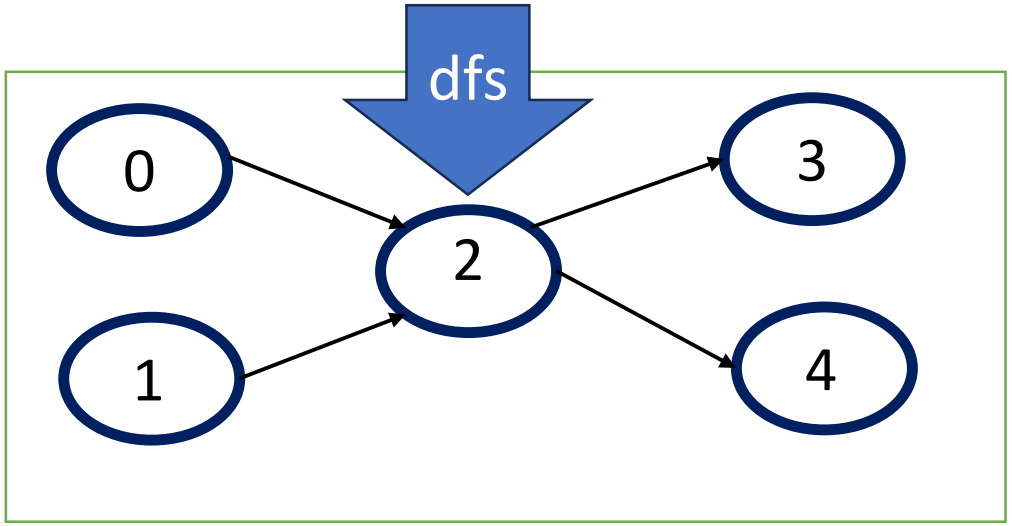
time = 1



Node	Finish Time
0	None
1	None
2	None
3	None
4	None

Computing Finish Time - Exercises

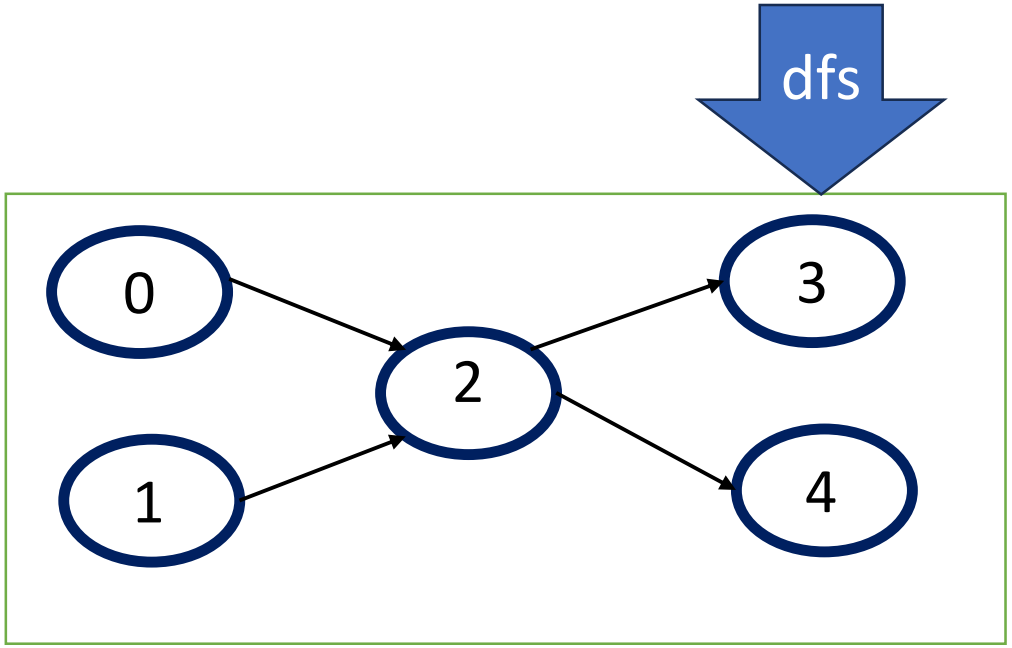
time = 2



Node	Finish Time
0	None
1	None
2	None
3	None
4	None

Computing Finish Time - Exercises

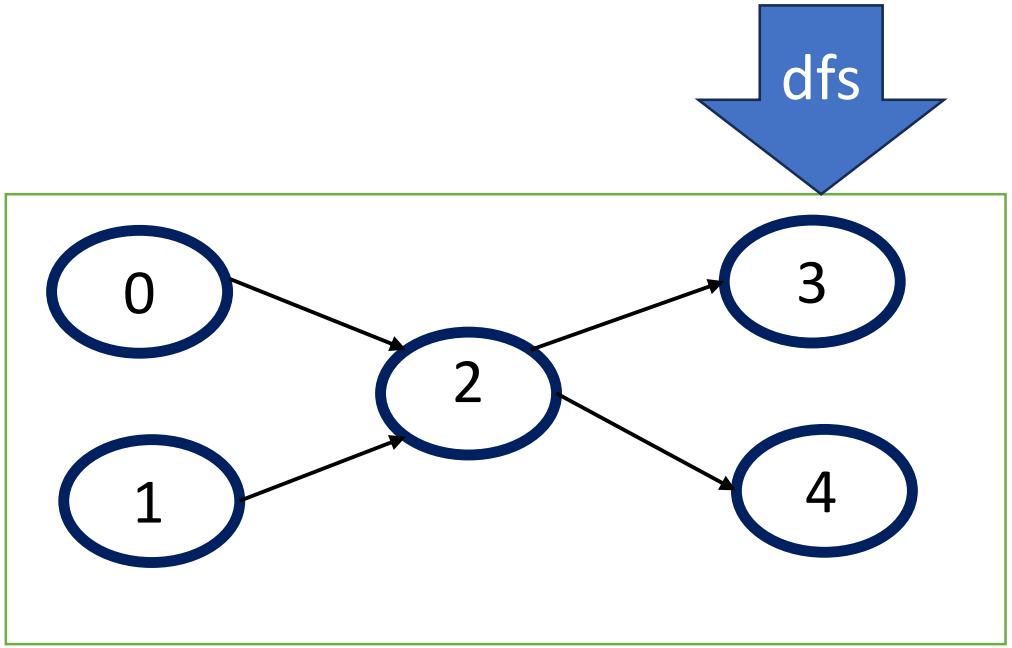
time = 3



Node	Finish Time
0	None
1	None
2	None
3	None
4	None

Computing Finish Time - Exercises

time = 4

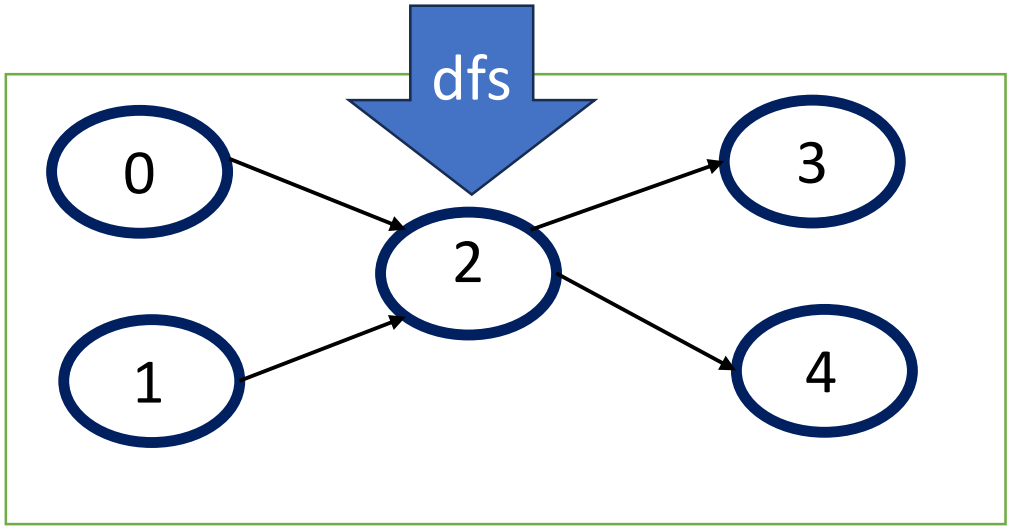


Node	Finish Time
0	None
1	None
2	None
3	4
4	None

Note: At time=4, we discover that node #3 doesn't have any unvisited neighbors. Therefore, FINISH TIME of node3 == 4

Computing Finish Time - Exercises

time = 4

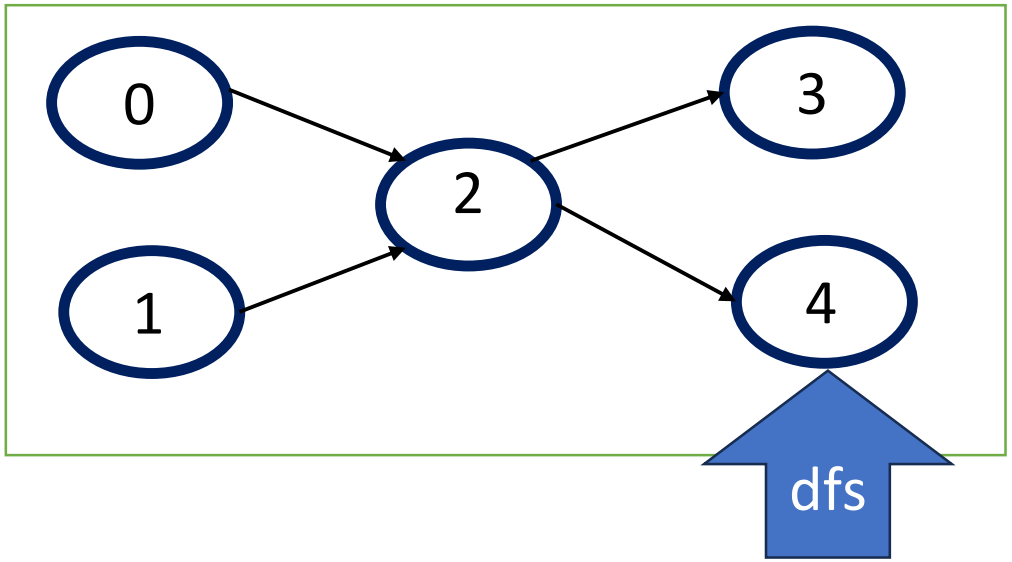


Node	Finish Time
0	None
1	None
2	None
3	4
4	None

Note: We go back to the previous node, assume that this operation takes 0 unit of time

Computing Finish Time - Exercises

time = 5

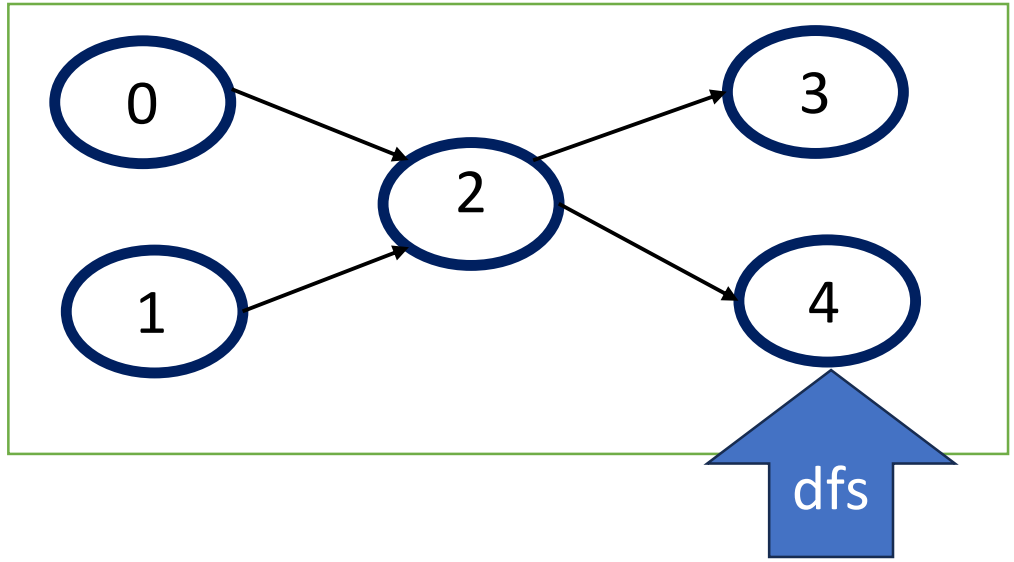


Node	Finish Time
0	None
1	None
2	None
3	4
4	None

Note: At time=5, we visit the next unvisited neighbor of node2.

Computing Finish Time - Exercises

time = 6

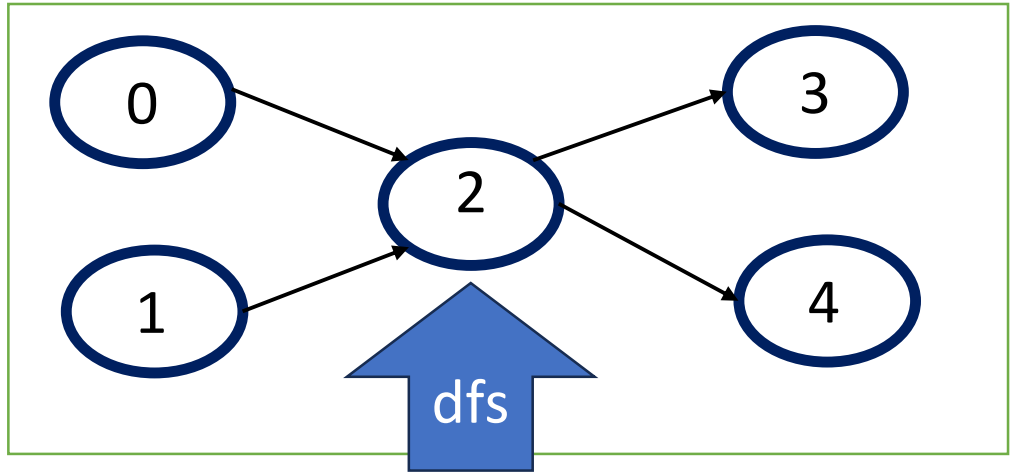


Node	Finish Time
0	None
1	None
2	None
3	4
4	6

Note: At time=6, we discover that node #4 doesn't have any unvisited neighbors. Therefore, FINISH TIME of node4 == 6

Computing Finish Time - Exercises

time = 6

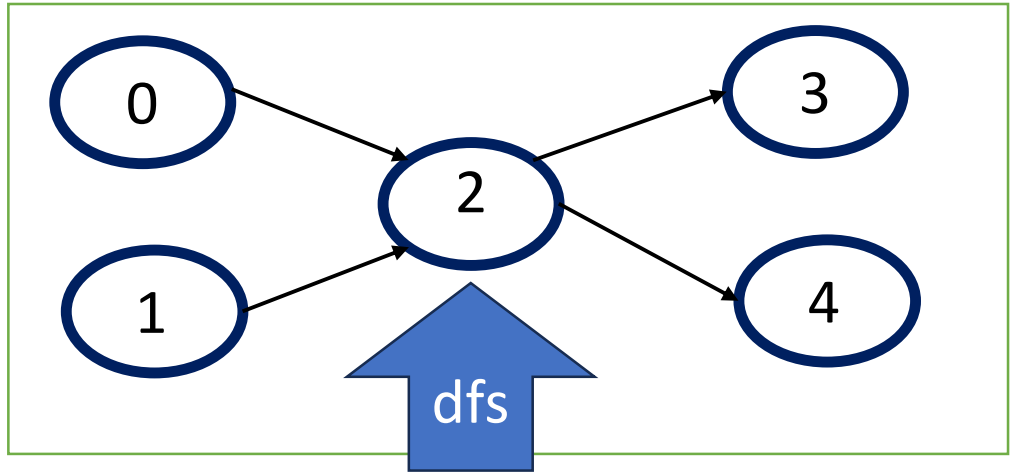


Node	Finish Time
0	None
1	None
2	None
3	4
4	6

Note: We move back to node2, time count is still 6.

Computing Finish Time - Exercises

time = 7

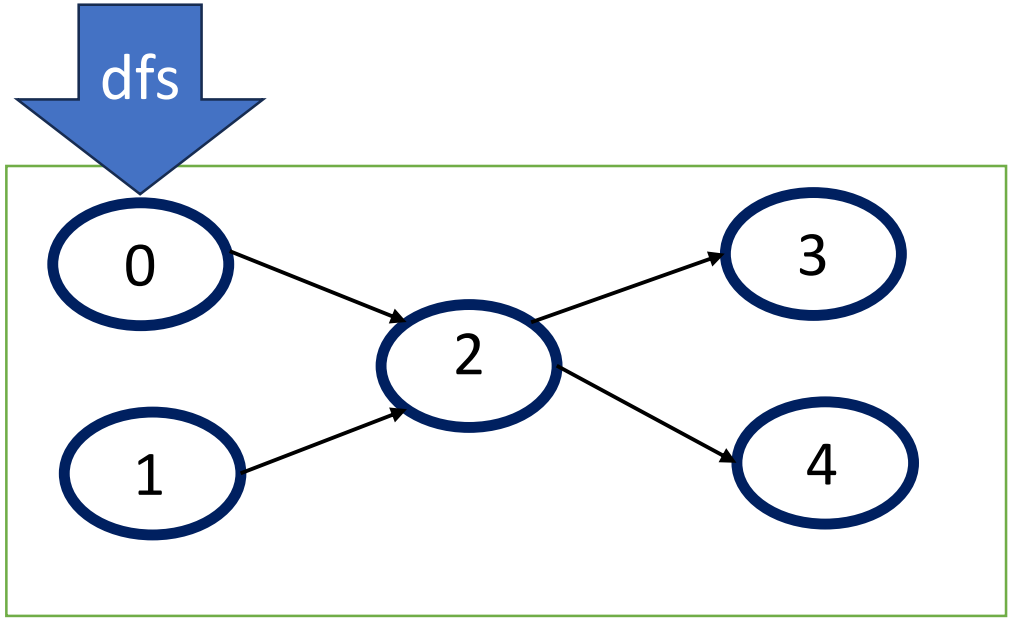


Node	Finish Time
0	None
1	None
2	7
3	4
4	6

Note: At time=7, we discover that node2 doesn't have any unvisited neighbors. Therefore, FINISH TIME of node2 == 7

Computing Finish Time - Exercises

time = 7

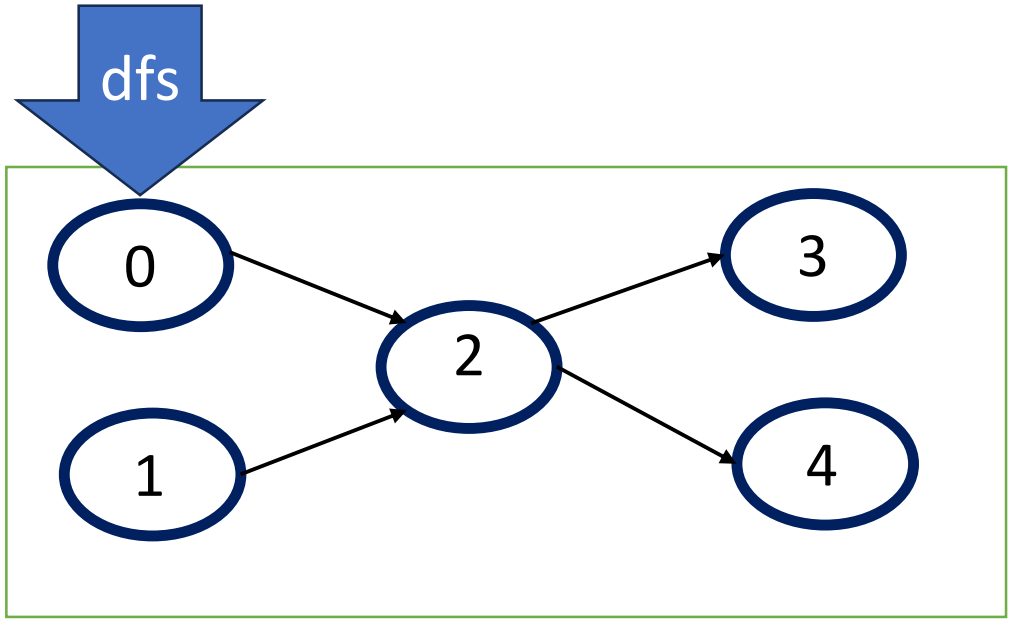


Node	Finish Time
0	None
1	None
2	7
3	4
4	6

Note: Then we move back to the previous node (node0). Time is still == 7

Computing Finish Time - Exercises

time = 8



Node	Finish Time
0	8
1	None
2	7
3	4
4	6

Note: At time=8, we discover that node0 doesn't have any unvisited neighbors. Therefore, FINISH TIME of node0 == 8

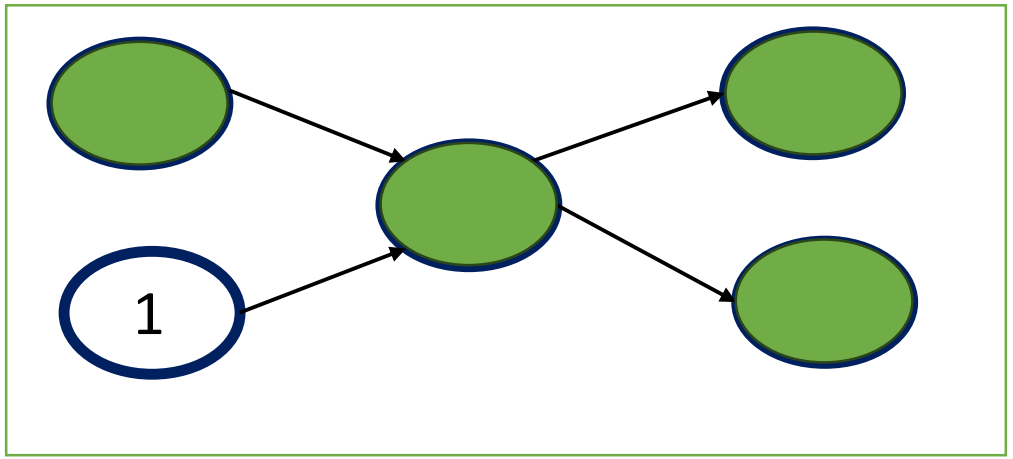
Computing Finish Time - Exercises

For each `node` in the graph:

If `node` is not visited, call **dfs(node)**

time = 9

dfs(1)



Node	Finish Time
0	8
1	None
2	7
3	4
4	6

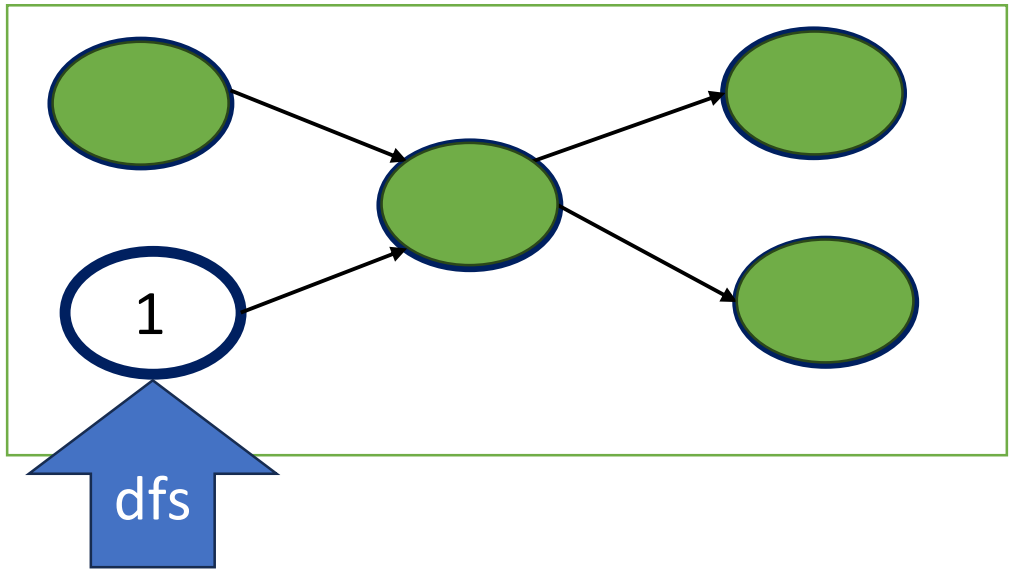
Note: At time=9, we call the dfs() method from node1, as that's still an unvisited node!

Computing Finish Time - Exercises

For each `node` in the graph:

If `node` is not visited, call **dfs(node)**

time = 10



Node	Finish Time
0	8
1	10
2	7
3	4
4	6

Note: At time=10, we discover that node1 doesn't have any unvisited neighbors. Therefore, FINISH TIME of node1 == 10

Computing Finish Time - Exercises

Node	Finish Time
0	8
1	10
2	7
3	4
4	6

- Finally, we have the DFS finish time for all nodes!
- Next? Sort the nodes based on their finish times.

Computing Finish Time - Exercises

Node	Finish Time
0	8
1	10
2	7
3	4
4	6

sort

Node	Finish Time
1	10
0	8
2	7
4	6
3	4

Topological sorting result: 1, 0, 2, 4, 3

Implementation in Python

iCollege > Coding Demo > TopologicalSorting.py

Attendance